FedECS-LT, from the problem to the defence

1 The problem

Federated learning. In federated learning, data stay on the devices or institutions that own them, the *clients*. A *server* coordinates training. In each *round* the server sends a model to the clients, each client trains it for a while on its own data, and the server combines the returned models into a new one. The standard way of combining is FedAvg: the server averages the clients' parameters, coordinate by coordinate. No raw data leave a client; only parameters or other summaries do.

Different models. FedAvg needs every client to train the same network, because only then do the j-th parameters of two clients mean the same thing. In practice clients run different hardware, so they may run different networks: a phone runs a small convolutional network, a hospital server a larger one. This is *model-heterogeneous* federated learning. Parameters of different architectures cannot be averaged, so the clients must share knowledge in another form. The paper studies one such form, a shared generator of synthetic features (Section 2).

Skewed and long-tailed labels. The second difficulty is in the labels. Two properties combine.

- *Label skew*: clients hold different classes in different proportions. A client may have hundreds of images of one class and none of another.
- *A global long tail*: over the whole federation, a few classes are frequent and many are rare.

The paper builds both in a controlled way. The class sizes follow

$$n_c \propto \rho^{-c/(C-1)}, \qquad c = 0, \dots, C-1,$$

so class 0 is the largest, class $C-1$ the smallest, and the *imbalance ratio* $\rho = n_0/n_{C-1}$ fixes how steep the tail is. The paper uses $\rho = 100$. Each class is then divided among the K clients by a Dirichlet draw: for class c a vector of shares $(p_{1c}, \dots, p_{Kc})$ is drawn from Dirichlet$(\alpha, \dots, \alpha)$, and client k receives a fraction p_{kc} of the class. A small concentration α gives very uneven shares; the paper uses $\alpha = 0.5$.

The running example. The text returns to one concrete case throughout: CIFAR-10-LT with $\rho = 100$, $K = 10$ clients and $\alpha = 0.5$, split with seed 3. The run logs give its class sizes,

$$(n_0, \dots, n_9) = (4800,\ 4800,\ 3620,\ 2170,\ 1301,\ 779,\ 467,\ 280,\ 168,\ 100),$$

18,485 training images in all. The profile asks for more than 4,800 images of classes 0 and 1, but CIFAR-10 has only 5,000 per class and 200 are held out for validation, so these two classes are capped. The realised ratio is therefore $4800/100 = 48$, not 100. The paper still names the setting by its target ratio $\rho = 100$ and gives the realised ratio in its dataset table (Table 1).

The rare class 9 shows what the split does to a rare class. Of its 100 images, client 7 holds 63. The other holders have between 1 and 14 each, and clients 2 and 5 have none. (Clients are

Table 1: The datasets as used in the paper (the paper's dataset table): number of classes, training images held by the clients, test images, and the realised ratio between the largest and the smallest class. The first two are the long-tailed benchmarks built with $\rho = 100$; the others keep their own class sizes.

Dataset	Classes	Training images	Test images	Largest / smallest class
CIFAR-10-LT	10	18,485	10,000	48
CIFAR-100-LT	100	10,926	10,000	96
DermaMNIST	7	6,231	2,005	75
OrganAMNIST	11	19,994	17,778	5
BloodMNIST	8	10,359	3,421	3
PathMNIST	9	19,997	7,180	2
EMNIST (ByClass)	62	19,970	116,323	20
FER-2013	7	19,997	3,589	17

numbered 1 to 10 here; in the run logs they are 0 to 9.) A class that is rare in the federation is held by few clients, and most of them see it only a few times.

How the loss is measured. Plain accuracy hides what happens to rare classes. Take a model that recognises classes 0–4 with recall 90% and classes 5–9 with recall 10%. On a test set with the training proportions above, its accuracy is

$$\frac{0.9 \cdot (4800 + 4800 + 3620 + 2170 + 1301) + 0.1 \cdot (779 + 467 + 280 + 168 + 100)}{18485}$$
$$= \frac{15021.9 + 179.4}{18485} \approx 82\%,$$

because the first five classes hold 90% of the images. Yet the model fails on half of the classes. *Balanced accuracy* is the mean of the per-class recalls, here $(5 \cdot 0.9 + 5 \cdot 0.1)/10 = 50\%$: every class counts equally, whatever its size. It is the paper's primary metric. The test sets of CIFAR-10 and CIFAR-100 are balanced (1,000 and 100 images per class), so there the two measures coincide; on the naturally imbalanced datasets they do not.

With ten client models, one per architecture, the paper reports the mean over the ten models of each model's balanced accuracy,

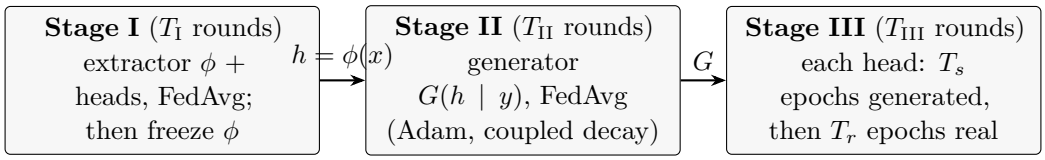
$$\text{balanced accuracy} = \frac{1}{G} \sum_{g=1}^{G} \frac{1}{C} \sum_{c=1}^{C} r_{gc},$$

where r_{gc} is the recall of class c by the model of architecture g and $G = 10$ is the number of architectures.

Two protocols. The paper tests its method in two settings.

- *Different client models*, the setting the method is built for: ten clients, each with its own head architecture, on the long-tailed benchmarks CIFAR-10-LT and CIFAR-100-LT and on six datasets whose classes are naturally imbalanced (Table 1).
- *One shared model*, the standard protocol of federated long-tail research: twenty clients train one ResNet-8 together. Here the specialised long-tail methods of the literature apply, so this setting shows how the method compares with them outside its own design.

The question the rest of this text builds on is how clients that run different models can share knowledge at all.



Every client sends parameters to the server in each round of every stage. Stage I averages the extractor over all clients (and each head over the clients of its architecture); Stage II averages the generator over all clients; Stage III averages each head over the clients of its architecture, which does nothing when every architecture has one client.

Figure 1: GeFL-F's three stages. The extractor and the generator are shared by all clients; the heads are not.

2 The base system: GeFL-F

Model-heterogeneous methods differ in what the clients exchange: predictions on shared inputs (FedDF, FedMD), class means of internal representations (FedProto, FedTGP), a shared part of the network (LG-FedAvg, FedGH, FedMRL), or a generator of synthetic data (FedGen, GeFL). The paper builds on GeFL-F, the feature-level variant of GeFL. It splits every client's network into two parts, a small *extractor* that all clients share and a *head* that is specific to the client's architecture, and lets the clients share a generator of extractor outputs. Training runs in three stages (Fig. 1).

Stage I: the shared extractor. All clients train a small convolutional extractor ϕ together with their heads by FedAvg, for $T_{\mathrm{I}} = 50$ rounds on CIFAR. The extractor is the same network for everyone, so it can be averaged. On CIFAR it is two 3×3 convolutions with three and ten output channels, each followed by batch normalisation and ReLU, and a 2×2 max pooling. A 32×32 image becomes a $10 \times 16 \times 16$ array, the *feature* $h = \phi(x)$, with $d = 2560$ numbers. After Stage I the extractor is frozen, and every client computes the features of its images once, without augmentation. From then on, every client works on features, not images.

The heads. Client k trains a head of its own architecture, one of ten convolutional networks CNN-1 to CNN-10 with one to four convolutional blocks of different widths. Each head has two parts. Its *body*, the convolutional blocks, maps a feature h to an *embedding* e; its *last layer* is linear and maps e to the C logits $f = We + b$. In the main setting there are ten clients and client k trains architecture k, so each architecture has exactly one client.

Stage II: the shared generator. The clients train a conditional variational autoencoder $G(h \mid y)$ on their features, again by FedAvg, for $T_{\mathrm{II}} = 200$ rounds, with the Adam optimiser and coupled weight decay (an L2 penalty $\lambda\|\theta\|^2/2$ added to the loss). The generator takes a class label y and random noise z and outputs a feature $\tilde{h}$ that should look like a real feature of class y. The label enters through a linear projection of the one-hot vector e_y; the weights that multiply class c form a vector r_c, the *class row*. The class rows are the only parameters of the generator that belong to one class: everything the generator knows that distinguishes class c from the others must pass through r_c.

Stage III: training the heads. For $T_{\mathrm{III}} = 100$ rounds, every client trains its head in two phases. In the *synthetic phase* it draws labels uniformly over all C classes, generates features for them, and trains for T_s epochs on these pairs with cross-entropy ($T_s = 1$ in GeFL-F). In the *real phase* it trains for $T_r = 5$ epochs on its own features. The server then averages the

heads of each architecture. With one client per architecture this averaging does nothing, so in the main setting the only knowledge a head receives from other clients comes through the generator. This fact returns in Sections 7 and 9.

Why a generator helps. Client 2 of the running example holds no image of class 9. Its real phase can only teach its head to separate the classes it holds. The synthetic phase can show the head features of class 9, made by a generator that was trained partly on client 7's 63 images of that class. If the generated class-9 features look like real ones, client 2's head learns class 9 without ever seeing a class-9 image. Whether they do look like real ones is the central question of Section 4.

What goes wrong. GeFL-F was designed and evaluated on balanced clients. Under the long tail of the running example it reaches 33.7 ± 1.2 balanced accuracy on CIFAR-10-LT, about the level of Local, in which every client trains alone (32.1 ± 0.7). On the tail classes, the 40% least frequent (classes 6–9), its recall is 13.9 ± 1.2. On CIFAR-100-LT it reaches 7.7 ± 0.4, against 7.9 ± 0.8 for Local. Federation through the generator brings almost nothing.

Where, inside these three stages, are the rare classes lost?

3 What federation can and cannot bias

Before locating the loss, one fact narrows the search. It is also the idea on which the whole method rests.

A client split is a pure label shift. The Dirichlet split of Section 1 decides *how many* images of each class a client gets, but within a class it picks the images uniformly at random. So the class-9 images of client 7 are a random subset of all class-9 images, and the same is true of every client and class. In symbols, the distribution of an image given its class is the same on every client as on the pooled data,

$$P_k(x \mid y) = P(x \mid y) \quad \text{for every client } k \text{ that holds class } y.$$

Only the class proportions $P_k(y)$ differ. The extractor is shared, so the same holds for the features $h = \phi(x)$.

The consequence: any quantity that describes one class, such as the mean feature of class 9, its spread, or the best weight vector for class 9 in a last layer, has the *same target* on every client that holds the class and on the pooled data. Federation cannot move that target. It can only make the estimate noisier, because each client sees fewer samples, or slower to learn. So when a class-level quantity ends up *biased* in a federated method, something in the method's estimation procedure must have caused it.

Sums are exact. Many class-level quantities can be computed from sums. The mean feature of class c is

$$\mu_c = \frac{\sum_k S_{kc}^{(1)}}{\sum_k n_{kc}}, \qquad S_{kc}^{(1)} = \sum_{i \in D_{kc}} h_i,$$

where D_{kc} is client k's set of class-c images and n_{kc} its size. If every client sends its count n_{kc} and its sum $S_{kc}^{(1)}$, the server obtains μ_c exactly, whatever the split. For class 9 of the running example, client 7 sends 63 and the sum of its 63 features, client 9 sends 14 and the sum of

its 14, and so on; the server divides the total sum by the total count, 100, and gets the same mean it would get from the 100 images in one place. The paper states this as a lemma.

Lemma (partition invariance)**.** *An estimator that is a function of a sum* $S = \sum_k \sum_{i \in D_k} T(x_i, y_i)$*, with* T *fixed in advance, takes the same value for every partition of the pooled data, and equals the centralised estimator.*

The proof is that addition does not depend on the order or grouping of its terms. The lemma is simple, and the paper says that it is known for closed-form federated classifiers (Fed3R, AFL, FedCOF). Its use here is that it tells which quantities a federated method can compute *without* the bias of Section 3: those that are functions of per-client sums. The class spread is one. With $S_{kc}^{(2)} = \sum_{i \in D_{kc}} \|h_i\|^2$,

$$V_c = \frac{\sum_k S_{kc}^{(2)}}{n_c} - \|\mu_c\|^2 = \mathbb{E}\big[\|h - \mu_c\|^2 \,\big|\, y = c\big],$$

the mean squared distance of a class-c feature from the class mean, is also exact.

Secure aggregation. Sending per-client sums to the server would reveal each client's class counts and feature sums. *Secure aggregation* is a cryptographic protocol that lets the server learn the sum over clients of vectors and nothing else about any single vector. The basic idea: every pair of clients agrees on a random mask; one adds it to its vector, the other subtracts it; each masked vector looks random to the server, but in the sum the masks cancel. The paper uses Flower's SecAgg+, a variant that tolerates clients dropping out.

The protocol works on integers, so each vector is divided by a public bound and quantised to 30 bits with unbiased stochastic rounding. The error is at most 1.5×10^{-8} of the bound per client and coordinate. Values beyond eight times the bound would be clipped, and with fewer than three participants the protocol cannot run and the sum is computed in clear. The paper calls the statistics *exact* in this sense: exact up to this quantisation. The bounds come from a small validation set that the server holds (200 images per class on CIFAR-10, held out from the training data), never from the clients' data.

What secure aggregation does not hide is the sum itself. If a class is held by a single client, the federation-level sum of that class is that client's sum. Section 5 returns to what the released sums reveal.

How bias still enters. If the targets cannot be biased by the split, a federated method can still bias its estimate of them in four ways, which the paper examines.

1. *The optimiser.* An optimiser can treat clients that lack a class differently from clients that hold it, and averaging can then pull a class parameter away from its target.

2. *Parameters tied to one client's prior.* A last-layer bias trained on one client's data learns that client's class proportions, which are not the federation's.

3. *What the synthetic data teach.* Training on generated features teaches whatever those features carry. If they do not carry their label, the training teaches something else.

4. *Sample size.* Nonlinear estimates, such as a spread or an inverse covariance matrix, are biased in small samples. Class 9 has 100 images and features of dimension 2560: its covariance matrix cannot even be inverted.

The paper does not claim that the list is complete. For example, drift of the shared extractor in Stage I, where the extractor is trained by FedAvg on skewed data, is not covered.

Which of these routes actually hurt GeFL-F?

4 Where GeFL-F loses the rare classes

The paper finds a candidate loss on each of the first three routes of Section 3: the optimiser in Stage II, the last layer in Stage III, and the synthetic data in Stage III. Each subsection below gives the intuition with the running example, the precise statement, and what the runs show. The strength of the evidence differs between the three, and the paper says so.

4.1 The generator: coupled weight decay under Adam

The situation. In one round of Stage II, every client starts from the global generator, takes some local Adam steps on its own features, and returns its copy; the server averages the copies. Consider the class row r_9 of the running example on client 2, which holds no image of class 9. Client 2 never generates a class-9 feature during training, so its data loss does not depend on r_9 at all. The only force on r_9 is the weight decay. With coupled decay the penalty $\frac{\lambda}{2}\|\theta\|^2$ is part of the loss, so the gradient of client 2's loss with respect to r_9 is exactly

$$g = \lambda r_9.$$

With plain SGD this would move r_9 by $\eta\lambda r_9$ per step, a tiny amount: with the generator's learning rate $\eta = 10^{-3}$ and decay $\lambda = 10^{-3}$, one part in a million of the row per step. Adam behaves very differently.

Why Adam makes the decay large. Adam divides each gradient coordinate by a running estimate of its own magnitude. With the running means m of the gradient and v of its square (decay rates $\beta_1 = 0.9$, $\beta_2 = 0.999$) and their bias-corrected versions $\hat{m}, \hat{v}$, a step is

$$\theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}.$$

Take one coordinate of r_9 equal to 0.5, so $g = \lambda \cdot 0.5 = 5 \times 10^{-4}$. On the first step $m = 0.1\, g$ and $v = 0.001\, g^2$; the bias corrections divide them by $1 - 0.9$ and $1 - 0.999$, so $\hat{m} = g$ and $\hat{v} = g^2$. The step is $\eta\, g/(|g| + \epsilon) \approx \eta = 10^{-3}$, in the direction of the sign of the coordinate. Now repeat with a decay 100 times smaller, $\lambda = 10^{-5}$: then $g = 5 \times 10^{-6}$, still much larger than $\epsilon = 10^{-8}$, and the step is again $\approx 10^{-3}$. Adam's step does not depend on the size of the gradient, only on its sign and consistency. A client that lacks class 9 therefore moves every coordinate of r_9 toward zero by about one learning rate per step, whatever the decay, 2,000 times faster than SGD would in this example.

Lemma (coupled decay under Adam)**.** *On a client without class c, the gradient of the loss with coupled L2 regularisation λ with respect to r_c is exactly λr_c. Since Adam is invariant to a positive rescaling of the gradient sequence up to its ϵ, each step of such a client moves every coordinate of r_c toward zero by about the learning rate η, independently of λ, as long as $\lambda|r_{c,j}| \gg \epsilon$. With $\lambda = 0$ this shrinkage vanishes; with decoupled weight decay (AdamW) it becomes a factor $1 - \eta\lambda$ per step, which is negligible here.*

Two remarks complete the picture. A client keeps its Adam state between rounds while its parameters are reset to the global ones, so a single step can be somewhat larger than η. And the effect is specific to *federated* training: on pooled data, every batch that contains class 9 gives r_9 a data gradient, Adam's estimate v of the squared gradient is dominated by these, and the decay stays small. This holds as long as class-9 batches come within the memory of v, about $1/(1-\beta_2) = 1000$ steps.

The holders do not compensate. One might expect the clients that hold class 9 to push r_9 back. They push it toward a good value, not toward a larger norm: their data gradient fits the row and need not lengthen it. They also see class 9 rarely. Stage II trains the generator for 5 local epochs per round in batches of 64 (from the code). Client 1 of the running example holds 4 class-9 images among 913, so at most 4 of its 15 batches per epoch, and at most 20 of its 75 steps per round, contain the class. On every other step its gradient on r_9 is again only the decay λr_9, and Adam shrinks the row exactly as on a client without the class. So most holders shrink a rare class's row most of the time.

Averaging turns this into contraction. After the round, the server averages the returned rows. Let $\Delta_k = \|r_{k,9}\| - \|r_9\|$ be the change of the row's length on client k, $\bar{G}$ the mean change over the holders, $\bar{D}$ the mean shrinkage over the non-holders, and $p = m/K$ the share of clients that hold the class.

Lemma (contraction under flat averaging)**.** *Under* $r_c^+ = \frac{1}{K}\sum_k r_{k,c}$, $\|r_c^+\| \le \|r_c\| + p_c\bar{G}_c - (1-p_c)\bar{D}_c$, *with equality if and only if all* $r_{k,c}$ *are non-negative multiples of one another. The row contracts whenever* $p_c < \bar{D}_c/(\bar{G}_c + \bar{D}_c)$.

The proof is the triangle inequality: the length of an average is at most the average of the lengths. The inequality is often strict, because clients that fit class 9 to their own few images move the row in different directions, and averaging vectors that point in different directions shortens them further. For a feel of the numbers: if eight holders lengthen the row by 0.02 each and two non-holders shorten it by 0.05, the bound allows a gain of $0.8 \cdot 0.02 - 0.2 \cdot 0.05 = 0.006$. If the holders' average change is zero, which is plausible when they rarely see the class, the row shrinks by at least $0.2 \cdot 0.05 = 0.01$ every round, over 200 rounds.

The paper also gives a predictor of the direction, counting Adam steps:

$$\kappa_c = \frac{\sum_{k \in H_c} q_k f_{kc}}{\sum_{k \notin H_c} q_k},$$

where H_c is the set of holders, q_k the number of local steps of client k, and f_{kc} the probability that a batch of client k contains class c. The numerator counts the steps on which a holder actually sees the class, the denominator the steps on which a non-holder decays it. When $\kappa_c < 1$ the row drifts toward zero. The paper is explicit that this is a heuristic, not a bound: it ignores the momentum Adam carries between rounds, and it was *not* checked against the measured rows.

The remedies. Two changes remove the effect. Decoupled weight decay (AdamW) applies the decay outside Adam's normalisation, so the shrinkage becomes the negligible factor $1 - \eta\lambda$. Or the rows can be averaged over their holders only, weighted by how much of the class each holds (class-count weighted averaging, CCWA, in Section 5.2): the non-holders' term $\bar{D}$ then drops out of the lemma.

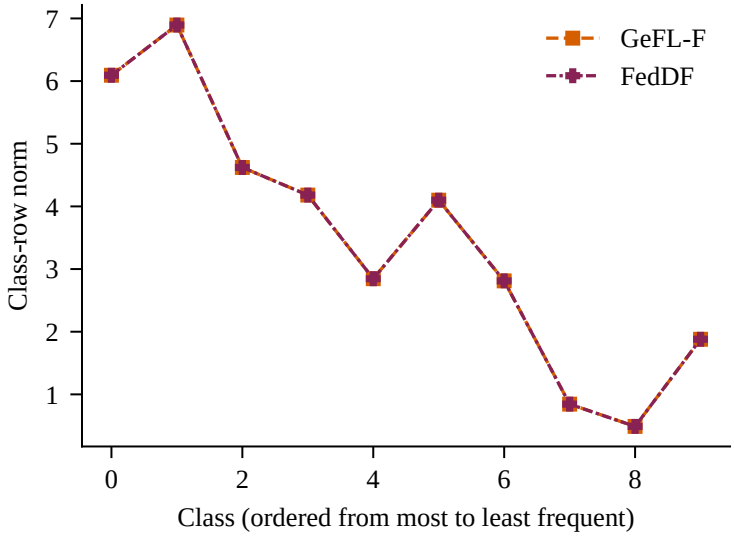


Figure 2: The paper's figure of the class rows: length of each class row of GeFL-F's generator after Stage II, classes from the most frequent (left) to the least frequent (right); CIFAR-10-LT, mean over seeds 3–5. Look at how the length falls toward the rare classes.

Table 2: The paper's fidelity table: the referee's per-class accuracy on generated features (%), averaged over all classes and over the head (30% most frequent), medium and tail (40% least frequent) classes; mean ± standard deviation over seeds 3–5. FedDF uses GeFL-F's generator. FedECS-LT's rows belong to Section 5.

Dataset	Generator	All	Head	Medium	Tail
CIFAR-10	FedECS-LT (generator)	25.9 ± 2.1	11.0 ± 2.2	16.7 ± 13.0	44.1 ± 5.0
CIFAR-10	FedECS-LT (after moment correction)	28.9 ± 4.6	23.5 ± 6.6	19.2 ± 12.4	40.3 ± 2.8
CIFAR-10	GeFL-F (generator)	12.9 ± 2.4	9.4 ± 0.9	11.2 ± 7.6	16.8 ± 0.3
CIFAR-10	FedDF (generator)	12.8 ± 2.2	9.2 ± 1.2	10.3 ± 7.2	17.3 ± 0.7
CIFAR-100	FedECS-LT (generator)	8.8 ± 1.2	5.9 ± 1.8	8.2 ± 0.8	11.5 ± 1.2
CIFAR-100	FedECS-LT (after moment correction)	8.8 ± 0.4	7.2 ± 0.8	9.0 ± 0.8	9.9 ± 0.6
CIFAR-100	GeFL-F (generator)	1.1 ± 0.1	1.1 ± 0.2	1.1 ± 0.2	1.2 ± 0.2
CIFAR-100	FedDF (generator)	1.1 ± 0.1	1.2 ± 0.2	1.0 ± 0.2	1.1 ± 0.2

What was measured. The paper's figure of the class rows (Fig. 2) shows that after Stage II the rows of the rarest classes are several times shorter than those of the most frequent ones. That is the predicted effect. It is not yet evidence that the shrinkage *causes* the rare classes' low accuracy, and the paper says so, for a reason that comes from a second measurement.

After Stage I the server trains a classifier, the *referee*, on the features of its validation images, and asks it to classify 300 generated features of each class. The share it assigns to the intended class is the generator's *fidelity* for that class. If the decay were what makes rare classes fail, the rare classes' fidelity would be low and the frequent classes' high. Instead GeFL-F's generated CIFAR-10-LT features are recognised at 12.9 ± 2.4 on average (chance is 10%), 9.4 ± 0.9 on the head classes and 16.8 ± 0.3 on the tail classes. On CIFAR-100-LT the mean is 1.1 ± 0.1, chance level (Table 2). GeFL-F's generator carries little class information for *any* class. The row shrinkage is a measured property of the generator; whether it causes the rare-class loss is open. The direct test is GeFL-F with AdamW and nothing else changed, one of the runs not yet returned (Section 7). The referee's accuracy on held-out real features was not logged, so the fidelity values compare generators with each other, not with real data.

4.2 The last layer: logit corrections cannot add information

The situation. In Stage III, client 2's head is trained on client 2's features only, in the real phase. Its last layer therefore learns client 2's class proportions: classes it holds often get large biases, the classes it lacks (class 9 among them) get pushed down by every cross-entropy step, because cross-entropy lowers the logits of all wrong classes. The natural fix from long-tail learning is to correct the loss with the client's class frequencies. *Logit adjustment* trains with $\ell(f + \tau \log \hat{\pi}_k,\, y)$, where $\hat{\pi}_k$ is client k's class distribution (with add-one smoothing, so that absent classes get a small non-zero frequency) and $\tau = 1$. Balanced Softmax and FedLC are of the same form: a constant vector a_k added to the logits during training.

Lemma (logit corrections are reparametrisations)**.** *Let a head have a free last-layer bias b, trained by SGD without decay on b, and let $a_k \in \mathbb{R}^C$ be a constant of client k, as in logit adjustment ($a_k = \tau \log \pi_k$), FedLC or Balanced Softmax. Training with the loss $\ell(f + a_k, y)$ from (θ_0, b_0) produces exactly the trajectory of training with $\ell(f, y)$ from $(\theta_0, b_0 + a_k)$: every parameter except b is the same at every step. FedRS, which scales the logits of absent classes by $\alpha \in (0, 1]$, is cross-entropy in the rescaled last-layer variables $\alpha w_c, \alpha b_c$ of those classes, trained with the smaller learning rate $\alpha^2 \eta$.*

The proof: $f + a_k = We + (b + a_k)$, and moving from b to $b + a_k$ is a translation, which does not change gradients. So training with the corrected loss is plain training that starts from the bias $b_0 + a_k$; at test time the correction is not applied, which amounts to subtracting a_k from the logits. This is exactly *post-hoc logit adjustment* of a head trained from a shifted starting bias.

A three-class example. A client holds 70 images of class A, 30 of class B and none of class C. With add-one smoothing its frequencies are $(71, 31, 1)/103$, so $a = \log \hat{\pi} \approx (-0.37, -1.20, -4.63)$. At test time the corrected head adds $+4.63$ to class C's logit relative to plain training, $+3.43$ more than to class B. That raises class C for *every* input by the same amount. It cannot make class C high for images of class C and low for others, because the weight vector w_C never received a single gradient that points toward class-C features: the client has no class-C features. The correction moves a threshold; it adds no information about the class. The body of the head can still learn a somewhat different representation, because the shifted logits change the gradients that reach it, but nothing tells it what class C looks like. A head trained on one client is an extreme case of what the literature calls minority collapse: for the classes the client lacks, the imbalance ratio is infinite.

What follows. A last layer that knows the absent classes needs information from other clients. The partition-invariance lemma of Section 3 says where to get it: a last layer that is a function of sums over all clients equals the one computed on pooled data. The paper's answer is the *exact readout* of Section 5.5, a classifier computed from summed embedding statistics.

What was measured. Two measurements support this part of the diagnosis, and they are the strongest evidence in the paper's analysis. First, replacing GeFL-F's trained last layers by the exact readout, with nothing else changed, raises its balanced accuracy by $+16.6$ points on CIFAR-10-LT and $+7.2$ on CIFAR-100-LT. Most of what GeFL-F loses is in the last layer. Second, the lemma predicts that Balanced Softmax, FedLC and FedRS differ only in their last layers. Once these are replaced by the same readout, the three methods reach 51.2 ± 0.7,

51.2 ± 0.7 and 51.1 ± 0.7 on CIFAR-10-LT, and 17.2 ± 0.3, 17.2 ± 0.3 and 17.2 ± 0.3 on CIFAR-100-LT: the same, as predicted.

4.3 The synthetic data: label-free features penalise the logit spread

The situation. In the synthetic phase, labels are drawn uniformly over the C classes and the generator makes a feature for each. If the generated feature carries its label, the head learns the class. If it does not, as the fidelity of Table 2 suggests for GeFL-F, the label is effectively independent of the feature, and the training target is the uniform distribution.

Lemma (label-free synthetic data)**.** *If the synthetic label y is uniform on $\{1, \dots, C\}$ and independent of the generated feature, then for every logit vector f,*

$$\mathbb{E}_y\, \ell(f, y) = \mathrm{LSE}(f) - \bar{f} = \log C + \mathrm{KL}(u \,\|\, \mathrm{softmax} f) = \log C + \frac{1}{2C} \sum_c (f_c - \bar{f})^2 + O(\|f - \bar{f}\|^3),$$

where $\mathrm{LSE}(f) = \log \sum_c e^{f_c}$*, $\bar{f}$ is the mean logit and u the uniform distribution.*

In words: averaged over the random label, the cross-entropy is smallest when all logits are equal, and it grows roughly with the variance of the logits. Training on label-free features therefore pulls the logits together. The paper measures this as the *logit spread*: the mean over images of the standard deviation of a head's logits across classes.

A numerical check with $C = 10$: equal logits give the loss $\log 10 = 2.303$. The logits $(2, 0, \dots, 0)$ give $\mathrm{LSE} = \log(e^2 + 9) = 2.797$ and $\bar{f} = 0.2$, so the loss is 2.597. The quadratic term alone predicts $2.303 + \frac{1}{20}(1.8^2 + 9 \cdot 0.2^2) = 2.483$; the rest is the higher-order term. The more confident the head, the larger the penalty, whatever the class.

What survives the alternation. The synthetic phase is followed by the real phase, which pushes the logits apart again for the classes the client holds. The paper analyses the alternation in a simplified model with quadratic losses (an appendix lemma). The round converges to a point where, along every direction of the parameters that the real loss leaves free, the synthetic loss alone decides. For a head trained on one client, the directions that concern its absent classes are almost free in this sense, so on them the label-free synthetic phase decides: it flattens exactly the logits the client most needs. Not exactly free, because cross-entropy on the real data still pushes the absent classes' logits down on data the head separates. Logit adjustment shifts those targets.

What was measured. The paper's figure of the logit spread (Fig. 3) shows, in every Stage-III round on CIFAR-10-LT, that the synthetic phase reduces the spread of the heads' logits on the clients' real features, and that it reduces it less for FedECS-LT than for GeFL-F. Two caveats keep this from being a clean test. FedECS-LT also differs from GeFL-F in its number of synthetic epochs, its soft targets and logit adjustment, so the comparison does not isolate the labels. And FedDF, which trains on GeFL-F's own label-free generator, loses *less* spread than FedECS-LT. So the size of the spread change is not determined by whether the generated features carry their label. The lemma is exact; how much it matters in the real pipeline is not settled.

What would a method look like that uses exact statistics exactly where these losses occur?

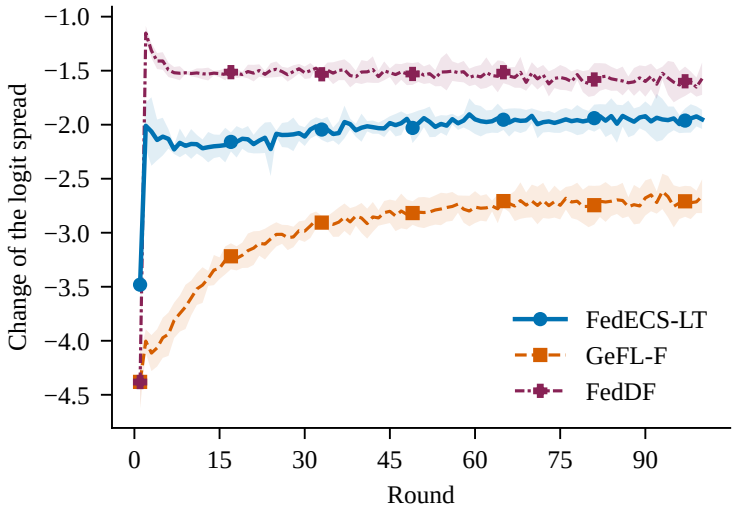


Figure 3: The paper's logit-spread figure: change of the logit spread during the synthetic phase of each Stage-III round (after minus before; negative means the logits were pulled together), CIFAR-10-LT, mean over clients and seeds 3–5. Look at the levels: GeFL-F lowest, FedECS-LT in between, FedDF highest.

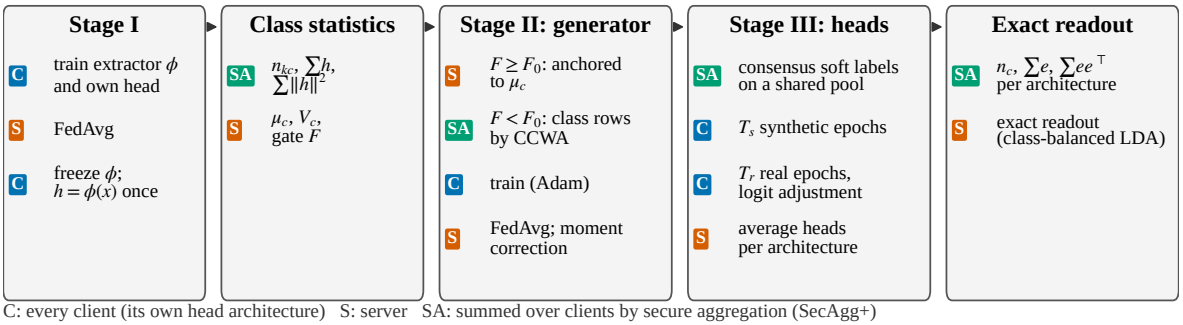


Figure 4: The paper's overview of FedECS-LT. C: computed by each client; S: computed by the server; SA: summed over the clients by secure aggregation. Read it left to right: statistics after Stage I, the generator chosen by the gate (Stage II), the moment correction, the heads (Stage III), and the exact readout last.

5 FedECS-LT, component by component

FedECS-LT (Exact Class Statistics for Long-Tailed federated learning) keeps GeFL-F's three stages and its frozen extractor. It changes how the class-specific quantities of the generator and the last layer are obtained, using statistics that secure aggregation computes exactly (Section 3) instead of parameters estimated by FedAvg. It also changes how the heads are trained, and it adds a last step that sets every last layer. Figure 4 is the paper's overview; this chapter takes the components in pipeline order.

5.1 Statistics release

What it answers. Every later component needs the class means and spreads of the features. They are functions of sums, so they can be computed exactly (Section 3).

How it works. After Stage I, client k computes for every class c its count n_{kc}, its feature sum $S_{kc}^{(1)}$ and its sum of squared feature norms $S_{kc}^{(2)}$. Secure aggregation returns the totals, and the server forms

$$\mu_c = S_c^{(1)}/n_c, \qquad V_c = S_c^{(2)}/n_c - \|\mu_c\|^2.$$

μ_c is the mean feature of class c and V_c its spread, the mean squared distance of a class-c feature from μ_c.

Cost. Each client sends $C(d+2)$ numbers once: for each class, one count, one number for the sum of squared norms, and d for the feature sum. On CIFAR-10 that is $10 \cdot 2562 = 25{,}620$ numbers.

5.2 The generator

The separation gate. The anchored generator below builds each class around its exact mean. That only helps if the means are far enough apart to tell the classes apart. The paper's appendix makes this precise: a generator of the form below separates two classes, in the Wasserstein distance between their generated distributions, by at most $(1 + L\|A\|_2)\|\mu_c - \mu_{c'}\|$, a constant times the distance between their means. So the server first measures how separated the means are, relative to the spread:

$$F = \frac{\frac{1}{C}\sum_c \|\mu_c - \bar{\mu}\|^2}{\frac{1}{C}\sum_c V_c}, \qquad \bar{\mu} = \frac{1}{C}\sum_c \mu_c.$$

The numerator is the spread of the class means around their centre, the denominator the average spread within a class. This is a trace form of Fisher's discriminant ratio. If $F \geq F_0 = 0.05$, the server uses the mean-anchored generator; otherwise it falls back to GeFL-F's generator, with class rows averaged by CCWA. The threshold was fixed during the method's development and not swept.

How small these numbers are is worth seeing. Over pairs of distinct classes, two class means are on average $\sqrt{2FC/(C-1)}$ root-mean-square spreads apart (an appendix identity). The running example has $F \approx 0.080$ (seed 3, from the run logs), so two CIFAR-10 class means are on average about $\sqrt{2 \cdot 0.080 \cdot 10/9} \approx 0.42$ spreads apart: in the frozen feature space, the classes overlap heavily. Across all the paper's runs F ranges from 0.014 to 0.449, and the gate chose class rows on CIFAR-10-LT with one shared model and FER-2013.

The mean-anchored generator. This is the default branch. It generates a class-y feature as

$$\tilde{h} = \mathrm{ReLU}(\mu_y + \mathrm{dec}(z,\, A\mu_y)), \qquad z \sim \mathcal{N}(0, I),$$

where dec is the decoder network and A a projection shared by all classes. The exact mean μ_y is added directly, and the decoder only has to model the deviation from it. The class enters only through μ_y, a number the server computed exactly, and through A, which every class shares. The generator therefore has no class row and no other class-specific parameter: the contraction of Section 4.1 has nothing to shrink. It is trained by FedAvg with Adam, like GeFL-F's.

The fallback: class rows with CCWA. When the gate finds the means too close, the generator keeps GeFL-F's class rows, but the server averages each row only over the clients

that hold the class, weighted by how much of it they hold:

$$r_c \leftarrow \frac{\sum_k E(n_{kc})\, r_{k,c}}{\sum_k E(n_{kc})}, \qquad E(n) = \frac{1-\beta^n}{1-\beta}, \quad \beta = 0.999.$$

$E(n)$ is the *effective number of samples*. It is zero for a client without the class, so non-holders drop out and the term $\bar{D}$ of the contraction lemma disappears. It is close to n for small n and saturates at $1/(1-\beta) = 1000$, so one large client cannot dominate a class. For class 9 of the running example, client 7 (63 images) gets weight $E(63) \approx 61.1$, client 1 (4 images) gets $E(4) \approx 4.0$, and clients 2 and 5 get zero. For class 1, client 7's 2,803 images give $E(2803) \approx 940$, not 2,803. Both sums are computed by secure aggregation, so the server never sees a single client's counts.

5.3 Moment correction

What it answers. Even a generator anchored to the right mean can produce features with the wrong spread, too concentrated or too dispersed, and a class-rows generator can drift in mean too. The server knows the exact mean μ_c and spread V_c, so it can correct the generated features to match them.

How it works. After Stage II the server draws 500 features per class from the generator, measures their mean $\tilde{\mu}_c$ and spread $\tilde{V}_c$, and from then on corrects every generated feature of class c by

$$\tilde{h} \leftarrow \mathrm{ReLU}\big(\mu_c + s_c(\tilde{h} - \tilde{\mu}_c)\big), \qquad s_c = \mathrm{clip}\Big(\sqrt{V_c/\tilde{V}_c},\ 0.1,\ 10\Big).$$

The map first moves the generated class to its exact mean, then rescales the deviations so that their spread becomes the exact one. A toy example: if a class's generated spread is a quarter of the real one, $\tilde{V}_c = V_c/4$, then $s_c = 2$ and every generated feature's deviation from the mean is doubled.

Why this map. Without the final ReLU and the clipping, this is the map that reaches the exact mean and spread while moving the generated features as little as possible (an appendix lemma; the proof is a Cauchy–Schwarz bound on $\mathbb{E}\langle T(X) - \mu_c,\, X - \tilde{\mu}_c\rangle$). The ReLU keeps features non-negative, as real features after a ReLU are, and the clipping prevents extreme rescaling, so in practice the moments match only approximately. Across every class and run of the long-tailed benchmarks, the corrected spread is 0.73 to 0.98 times the exact one (median 0.86).

Cost. The server sends $(\mu_c, \tilde{\mu}_c, s_c)$ to the clients once; each client holds the generator and corrects its own samples.

What the evidence says. Matching moments is not the same as making features recognisable. The fidelity table (Table 2) shows that on CIFAR-10-LT the correction raises the referee's accuracy on the head classes from 11.0 ± 2.2 to 23.5 ± 6.6 and lowers it on the tail classes from 44.1 ± 5.0 to 40.3 ± 2.8. Whether it improves accuracy is a question for the component ablation, which has not returned yet.

5.4 The heads

Each Stage-III round, client k first takes $T_s\lfloor n_k/B \rfloor$ SGD steps on batches of B corrected generated features ($B = 128$ on CIFAR), that is, T_s epochs of the size of its real data, and then

trains $T_r = 5$ epochs on its real features. Each phase starts a fresh SGD optimiser (learning rate 0.1, no momentum). Three things differ from GeFL-F.

Logit adjustment in the real phase. The real phase uses $\ell(f + \tau \log \hat{\pi}_k, y)$ with $\tau = 1$ and the client's add-one smoothed class frequencies $\hat{\pi}_k$. Section 4.2 showed that this adds no information about absent classes to the last layer, and the method still uses it, for two reasons the paper gives. Without it, cross-entropy keeps pushing the absent classes' logits down during the real phase (Section 4.3). And it can change what the body learns. Note that $\hat{\pi}_k$ is the client's own frequencies, not a secure statistic. How much logit adjustment contributes is one of the pending component ablations.

Consensus soft labels in the synthetic phase. The synthetic phase still draws labels uniformly, but the target is no longer the one-hot label alone. For a generated feature $\tilde{h}$ with label y,

$$t(\tilde{h}) = (1 - \omega)\, e_y + \omega \, \frac{1}{G} \sum_{g=1}^{G} \mathrm{softmax}\big(f_g(\tilde{h})\big), \qquad \omega = 0.5,$$

where e_y is the one-hot vector of y and f_g the current head of architecture g. The second term is the federation's averaged prediction on that feature, the target form used by AvgKD and FedDF. If a generated "class 9" feature looks to the federation's heads like class 3 with probability 0.4 and like class 9 with 0.05, its target puts $0.5 + 0.5 \cdot 0.05 = 0.525$ on class 9 and 0.2 on class 3. The target follows what the feature actually carries instead of forcing a label it may not support, and every head learns from the ensemble of all ten architectures.

The average is computed by one secure sum. Every client regenerates the same pool of generated features from a seed the server sends, and returns $w_k \, \mathrm{softmax}(f_{a(k)}(\tilde{h}))$, with $w_k = 1/(G \cdot |\{j : a(j) = a(k)\}|)$, the inverse of the number of architectures times the number of clients of its own architecture. The secure sum of these is the average above. For this every client receives every architecture's current head, which FedAvg within an architecture would not reveal; secure aggregation hides only each client's predictions from the server.

Ten synthetic epochs. $T_s = 10$ instead of GeFL-F's 1. This is more computation, not a statistic, and the paper lists it as one of the method's changes.

5.5 The exact readout

This is the most important component, because the results will show that it carries most of the gain.

What it answers. Section 4.2 showed that a last layer trained on one client knows nothing about the classes the client lacks, and that a last layer computed from sums over all clients equals the one computed on pooled data. The exact readout is such a last layer: after Stage III, every architecture's trained last layer is replaced by a classifier computed from aggregated statistics of the embeddings.

The classifier. Suppose the embeddings e of each class c are Gaussian with mean ν_c and a covariance Σ shared by all classes, and every class is equally likely. Then the most probable class for an embedding e maximises

$$\log p(e \mid c) = -\tfrac{1}{2}(e - \nu_c)^\top \Sigma^{-1} (e - \nu_c) + \text{const} = e^\top \Sigma^{-1} \nu_c - \tfrac{1}{2} \nu_c^\top \Sigma^{-1} \nu_c - \tfrac{1}{2} e^\top \Sigma^{-1} e + \text{const}.$$

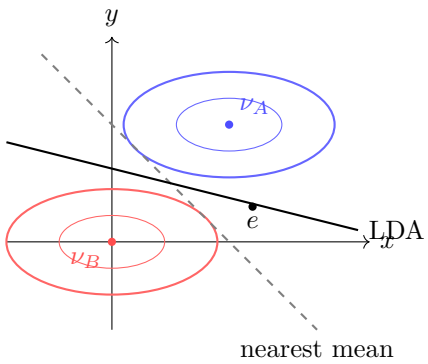


Figure 5: Illustration of the readout's decision rule (made-up numbers, not data). Ellipses: equal-density contours of two classes with a shared covariance that is narrow in y. Dashed: the nearest-mean boundary $x + y = 1$. Solid: the LDA boundary $x + 4y = 2.5$. The point e is on A's side of the dashed line but on B's side of the solid one.

The term $-\frac{1}{2}e^\top \Sigma^{-1} e$ is the same for every class, so it does not affect which class wins. What is left is linear in e: the logit of class c is $w_c^\top e + b_c$ with

$$w_c = \Sigma^{-1}\nu_c, \qquad b_c = -\tfrac{1}{2}\,\nu_c^\top \Sigma^{-1}\nu_c.$$

This is linear discriminant analysis (LDA). The usual LDA adds $\log \pi_c$, the class prior, to b_c, which favours frequent classes. Leaving it out makes the classifier *class-balanced*: it treats every class as equally likely, which is what balanced accuracy rewards. The classifier is the nearest class mean in the Mahalanobis distance $(e-\nu_c)^\top \Sigma^{-1}(e-\nu_c)$, a distance that measures each direction in units of the spread along it.

A two-dimensional example. Take two classes with means $\nu_A = (1, 1)$ and $\nu_B = (0, 0)$ and the shared covariance $\Sigma = \mathrm{diag}(1,\, 0.25)$: the second coordinate is four times less noisy than the first. Then $\Sigma^{-1} = \mathrm{diag}(1, 4)$, $w_A = (1, 4)$, $b_A = -\frac{1}{2}(1 + 4) = -2.5$, $w_B = 0$, $b_B = 0$, and class A wins when $x + 4y > 2.5$. The nearest-mean rule would instead decide by $x + y > 1$. For the point $e = (1.2,\, 0.3)$ the nearest-mean rule says A, since $1.5 > 1$, but LDA says B, since $1.2 + 1.2 = 2.4 < 2.5$. The point is far from A along the precise coordinate and close to B there, and LDA trusts that coordinate more (Fig. 5).

Shrinkage. In practice Σ is estimated, and the embedding dimension d_e is large: from the code, d_e ranges from 80 (CNN-10) to 1024 (CNN-1). An estimated covariance in high dimension has small eigenvalues that are too small, and inverting it amplifies noise. The readout therefore uses

$$\Sigma_\gamma = (1-\gamma)\Sigma + \gamma\,\frac{\mathrm{tr}\,\Sigma}{d_e}\,I, \qquad \gamma = 0.1,$$

a mix of the estimate with a multiple of the identity that has the same average variance. In the example above $\Sigma_\gamma = 0.9\,\mathrm{diag}(1, 0.25) + 0.1 \cdot 0.625\,I = \mathrm{diag}(0.9625,\, 0.2875)$: the two directions are pulled toward each other, and the inverse is stable. The paper's version also uses a pseudo-inverse, and sets $b_c = -10^4$ for a class that no client holds, so that it is never predicted.

How it is computed from sums. After Stage III the server sends every architecture's body to every client. Each client passes its real features through every body and computes, per architecture, its per-class counts, the per-class sums $\sum e$ of the embeddings, and the second

moment $\sum ee^\top$. Secure aggregation sums them. From these the server gets the class means ν_c and the pooled within-class covariance, which is the total second moment minus $\sum_c n_c \nu_c \nu_c^\top$, divided by the number of images minus the number of classes. Everything is a function of sums, so by partition invariance the readout equals the same estimator computed centrally on the pooled embeddings of the same body. It is not the same as a last layer trained centrally by SGD.

What it is and is not. With the true means and covariance and $\gamma = 0$, and if the embeddings really were Gaussian with a shared covariance, this would be the Bayes classifier for equally likely classes. Computed from data with shrinkage, it is a plug-in estimate of that classifier. When the embeddings are not Gaussian with a shared covariance, which is the usual case, it is a linear classifier with no optimality guarantee. It belongs to a known family of closed-form federated classifiers: Fed3R (a ridge-regression classifier), FedNCM (nearest class means) and FedCOF (an LDA-type classifier with a covariance estimated from class means). What the paper adds is computing it *per architecture* with statistics from *all* clients, including those of other architectures. That is possible only because every client runs every architecture's body.

Cost. One extra round. Each client sends $C(d_e+1)+d_e^2$ numbers per architecture: C counts, C class sums of length d_e, and the $d_e \times d_e$ second moment. For CNN-1 with $d_e = 1024$ the second moment alone is about a million numbers. Every client must run a forward pass (no training) of every architecture's body on its own features once; a client that cannot do so would be left out of the sums.

It can be added to other methods. The readout uses only a head's body, never its trained last layer, so it can be added to any method whose heads have a body and a linear last layer. The paper uses this to separate the readout's gain from the rest of the method: it reports every method of the GeFL-F pipeline both without the readout and with it added ("ER"). This is the comparison on which Section 7 turns.

5.6 The whole recipe

The paper's Algorithm 1 in words, one run:

1. **Stage I**: train the extractor ϕ and the heads by FedAvg; freeze ϕ; every client computes $h = \phi(x)$.
2. **Statistics release**: clients send $n_{kc}, S_{kc}^{(1)}, S_{kc}^{(2)}$ by secure aggregation; the server forms μ_c, V_c and the gate F.
3. **Stage II**: if $F \geq F_0$, train the anchored generator by FedAvg; otherwise GeFL-F's generator with class rows averaged by CCWA.
4. **Moment correction**: the server measures $\tilde{\mu}_c, \tilde{V}_c$ and sends $(\mu_c, \tilde{\mu}_c, s_c)$.
5. **Stage III**, for T_{III} rounds: the server sends the heads and a pool seed; the clients return their predictions on the pool by secure aggregation, giving the consensus soft labels; each client trains T_s epochs on corrected generated features with consensus soft labels, then T_r epochs on real features with logit adjustment; the server averages heads of the same architecture.

6. **Exact readout**: clients send embedding sums per architecture by secure aggregation; the server sets every last layer to the class-balanced LDA.

The settings are the same on every dataset: $F_0 = 0.05$, $\beta = 0.999$, 500 samples for the moment correction with $s_c \in [0.1, 10]$, $T_s = 10$, $T_r = 5$, $\tau = 1$, $\omega = 0.5$, $\gamma = 0.1$; the generator uses Adam with learning rate 10^{-3} and coupled decay 10^{-3}, as in GeFL-F.

Which component uses what. Only three components use the secure statistics: the anchored generator (or CCWA), the moment correction, and the readout. Logit adjustment uses the client's own frequencies, consensus soft labels use the heads' predictions, and $T_s = 10$ is computation. The method's name describes its first group.

5.7 Cost and privacy

Beyond GeFL-F, FedECS-LT sends the class statistics once, the moment correction once, one vector of C consensus probabilities per generated feature per round, and the readout sums once. The measured totals are in Section 7.

What is revealed? Secure aggregation hides each client's own vectors. The server, and through the generator and the moment correction every client, learns the federation-level class means and spreads. For a class that one client holds, these equal that client's own statistics. For a class with few images they reveal more about each image than a mean over thousands would: each of class 9's 100 images moves its class mean by a hundredth of its own value. The features come from a shallow extractor, two convolutions and a pooling, so a class mean is close to a mean image at half resolution. The paper did not test how much of the images an inversion of the released means recovers. Formal differential privacy, adding calibrated noise to the sums, is possible but not part of the results, and it would not cover the FedAvg training of the generator and heads. The paper therefore says FedECS-LT should not be called privacy-preserving beyond what secure aggregation guarantees against an honest-but-curious server. Every client also receives every architecture's head (consensus) and body (readout), so the architectures are not secret.

How is such a method tested, and against what?

6 The study design

The experiments have to show three things: that the losses of Section 4 occur, that FedECS-LT recovers them, and which part of FedECS-LT does so. Every choice below serves one of these, or protects the comparison against a way of being unfair.

6.1 The research questions

- **RQ1, diagnosis.** Do the three losses occur in GeFL-F's own runs? Answered by the class rows, the readout gain and the logit spread (Section 4).
- **RQ2, effect.** Does FedECS-LT, ending with its readout, raise balanced accuracy over GeFL-F and the other model-heterogeneous methods, each as it is without the readout? Answered by the main comparison (Section 7.1).

- **RQ3, body or last layer.** How much of the difference remains when every method of the GeFL-F pipeline gets the same readout, so that only the heads' bodies differ? Answered by the readout comparison (Section 7.4).
- **RQ4, scope.** Does the effect hold with natural class imbalance and in the protocol with one shared model? Answered in Sections 7.6 and 7.7.
- **RQ5, components.** Which changes carry the effect? Answered by the ablations, whose runs are pending (Section 7.10).

RQ3 is the question that decides how the paper's claim must be worded, as Section 7 shows.

6.2 Datasets and splits

The long-tailed benchmarks. CIFAR-10-LT and CIFAR-100-LT are built by the paper with $\rho = 100$. GeFL-F uses half of CIFAR-10's training set, so the profile is applied to that half (and to 22% of CIFAR-100, which gives the usual CIFAR-100-LT shape). As Section 1 showed, the largest classes are capped, so the realised ratios are 48 and 96 (Table 1). The class index orders classes from the most to the least frequent, so the tail classes are the same in every run. MNIST, Fashion-MNIST and SVHN belong to the same benchmark set; their runs had not returned when this text was written.

The naturally imbalanced datasets. Six datasets keep their own class sizes, with training sets capped at 20,000 images: DermaMNIST, OrganAMNIST, BloodMNIST and PathMNIST (medical images), EMNIST ByClass (handwritten characters, 62 classes) and FER-2013 (facial expressions). Their realised ratios range from 2 to 75. On OrganAMNIST, BloodMNIST and PathMNIST the ratio is 5 or less, so there the results speak to label skew more than to a long tail. These six datasets were never used while the method was being developed, which makes them the cleanest evidence in the paper.

Splits. Each class is divided among $K = 10$ clients by a Dirichlet draw with $\alpha = 0.5$. The draw is repeated, at most 100 times, until every client holds at least ten images. A held-out block of training images that no client holds, 200 per class on CIFAR-10, forms the server's validation set. It has two uses only: it trains the referee of the fidelity measurement, and it sets the public bounds of secure aggregation. It is never used to select a model or a setting. Every reported number is on the official test set. The paper checked that every client's class counts are identical across methods for every dataset and seed.

Seeds. Every configuration runs with seeds 3, 4 and 5. A seed fixes the split and every method's initialisation, so two methods with the same seed see exactly the same clients: their results are *paired*.

6.3 The two protocols

Different client models. The main setting is GeFL-F's: 10 clients, one per architecture, with GeFL-F's rounds ($T_{\mathrm{I}}, T_{\mathrm{II}}, T_{\mathrm{III}} = 50, 200, 100$ on CIFAR). All federated methods of this setting share the same Stage-I extractor for a given dataset and seed, so they differ only in what they do after Stage I.

One shared model. The second protocol is the one used by federated long-tail research, taken from the FedLTLib benchmark, whose settings come from CReFF: CIFAR-10-LT and CIFAR-100-LT at $\rho = 100$, 20 clients, 8 of them (40%) in each round, Dirichlet $\alpha = 0.5$, one ResNet-8 shared by all, 200 rounds of 10 local epochs (batch 32, SGD with learning rate 0.1, momentum 0.9, decay 0.99 per round). The baselines run their authors' LongTailLib code unchanged on the paper's splits, with ResNet-8 for every method.

FedECS-LT is fitted into this protocol by splitting the ResNet-8 after its second stage into a frozen extractor and a head. The extractor trains for 100 rounds, the generator for 100, and the head for 100 with 10 real epochs per round; so FedECS-LT uses the baselines' 200 training rounds plus 100 generator rounds. Two differences matter later: its head trains on frozen features of unaugmented images, while the baselines train the whole network end to end with augmentation; and with a single architecture the consensus soft label is just the shared model's own prediction.

6.4 The methods compared

Each group of methods answers a specific question.

- **GeFL-F**, the method FedECS-LT extends: the difference to it is the effect of the changes.
- **FedDF**, ensemble distillation on the server: after each round the server distils every architecture toward the ten architectures' mean prediction, for 20 steps. FedDF originally distils on public data; here it distils on GeFL-F's generated features, so that no method uses data that no client holds. It is the strongest generator-based competitor.
- **Per-client long-tail losses**: Balanced Softmax, FedLC and FedRS, applied to the heads with *no generator*. They test whether correcting the loss alone repairs the long tail. By the reparametrisation lemma they should behave alike, and Section 4 showed that they do once the readout is added.
- **Model-heterogeneous methods from the HtFLlib library**: prototypes (FedProto, FedTGP), a generator of embeddings (FedGen), a shared global head (FedGH), shared last layers (LG-FedAvg) and shared sub-networks (FedMRL). They represent the other ways of sharing knowledge between different models. They were designed for label skew without a global long tail and are ported to the frozen extractor with HtFLlib's settings.
- **Local**: each client trains its extractor and head alone. It is the reference without federation.
- **One-shared-model methods**: FedAvg, FedProx, MOON, FedLC, CCVR, CReFF, FedETF, Fed-GraB, FedLoGe, FedYoYo and RUCR, in the second protocol only.

Two further groups, GeFL's own grouped baselines and a reference trained on all data pooled, are planned for GeFL's 50-client setting and had not returned when this text was written.

Fairness of the comparison. The paper's fairness table records what each method spends and exchanges. FedECS-LT trains 10 synthetic epochs per round against 1 for GeFL-F and FedDF and none for the methods without a generator; it exchanges consensus soft labels; and it ends with the readout. GeFL-F, FedDF and the three losses are also reported with the readout added. The HtFLlib methods are not, so FedECS-LT's margin over them includes the readout's gain. No baseline was tuned on the paper's data.

6.5 Metrics

The primary metric is balanced accuracy averaged over the ten client models (Section 1). The others each answer a narrower question:

- **tail recall**, the mean recall of the 40% least frequent classes (classes 6–9 of CIFAR-10): are the rare classes recovered?
- **worst-class recall**, the lowest class recall after averaging over the models: is any class abandoned?
- **classes with zero recall for every model**: how many classes does the federation fail completely?
- **accuracy on each client's own class mix**: the test accuracy re-weighted to the class frequencies of the model's own client. It measures how well a model serves its own client, which balanced accuracy deliberately ignores.
- **expected calibration error** (15 bins): do the predicted probabilities match the observed frequencies?
- **the weakest architecture's balanced accuracy**: does the mean over ten models hide a poor one?
- **GeFL's measure**, each model's best test accuracy over the evaluated rounds, given in the appendix for comparison with GeFL's paper. The main tables report the model after the last round, which involves no selection on the test set.
- **cost**: communication per client, wall-clock time (on a shared T4 GPU, so only indicative) and floating-point operations.

6.6 Statistics

Paired differences. A difference between FedECS-LT and a method is the mean of the per-seed differences over the seeds both have, with a 95% t interval and a paired t-test. Pairing removes the variation that comes from the split, because both methods see the same split on each seed.

A worked example with the run logs: with the readout, FedECS-LT minus Balanced Softmax on CIFAR-10-LT is 0.65, 1.30 and 2.34 points on seeds 3, 4 and 5. The mean is 1.43 and the standard deviation 0.86. The t statistic is $1.43/(0.86/\sqrt{3}) = 2.89$ with 2 degrees of freedom, giving an uncorrected two-sided $p = 0.10$. The 95% interval is $1.43 \pm 4.30 \cdot 0.86/\sqrt{3}$, which is [-0.7, +3.6]: it contains zero.

Holm's correction. A table compares FedECS-LT with many methods at once, and with many tests some will look significant by chance. Holm's procedure controls this. Sort the m p-values from smallest to largest, multiply the smallest by m, the next by $m - 1$, and so on, and keep each adjusted value at least as large as the one before. A difference is called significant only if its adjusted p is below 0.05. The family is every other method of the same table and setting. In the example, after correction over the five methods of the pipeline, Balanced Softmax's p becomes 0.37.

Why three seeds are weak. Three paired differences give little power. A sign-flip permutation test, which assumes nothing about the distribution, cannot go below $p = 0.25$ with three pairs: of the $2^3 = 8$ ways to flip the signs, the observed pattern and its mirror are always 2 of the 8. The t-test reaches smaller values only because it assumes normally distributed differences. A result near $p = 0.05$ can also depend on which methods are in the Holm family.

The noise floor. GPU computation is not bit-reproducible, so two runs with the same seed differ. The paper measured this by running the BloodMNIST notebook twice with the same run list (39 runs). The standard deviation of the difference between the two runs of one configuration is 1.2 points as trained and 0.6 with the readout. For a mean of three paired differences that gives about half of these values, and the paper treats differences smaller than twice that, 1.4 points as trained and 0.7 with the readout, as unresolved. Exceeding the floor is necessary, not sufficient: a difference above it counts as resolved only if its Holm-corrected test is also significant.

6.7 What the development saw

The settings of FedECS-LT were fixed during its development, before the reported runs. The development used seeds 0 to 5, including seeds 3 to 5 with the same client splits as the reported runs, and every run was evaluated on the same official test sets. It compared about 36 configurations of the method and its variants on CIFAR-10-LT and 17 on CIFAR-100-LT, plus runs on MNIST, Fashion-MNIST and SVHN. It never used the six naturally imbalanced datasets. The baselines were not tuned. So on the CIFAR benchmarks the reported numbers may favour FedECS-LT through this selection, which is why the natural datasets matter, and why a run on new seeds 6–8 was added (pending).

6.8 Threats to validity

The paper lists these threats in its design section. Each one changes how a result may be read.

- **Compute.** FedECS-LT trains longer and communicates more. Its gain is therefore not shown to come from the statistics rather than from the extra computation until the matched-epoch runs return.
- **Missing runs.** Some settings have fewer than three seeds (EMNIST, and CIFAR-100-LT with one shared model), and those values carry no interval. With one shared model some runs of FedECS-LT failed where the baselines finished, so the missing runs are not random.
- **Ports.** The HtFLlib methods run on a frozen extractor with settings meant for end-to-end training, which may not suit them.
- **The mean over models.** Balanced accuracy averages ten models of different sizes, so a large gain on small heads and a small one on large heads give the same number; the weakest architecture is reported for this reason.
- **The readout comparison.** The readout is the same linear classifier on every body, so accuracy with it measures the bodies only through that classifier.

- **The validation set.** The server holds a small validation set for the diagnostics and the bounds; a deployment without such data would need another source for the bounds.
- **The long tail is synthetic** on the benchmarks, with the same tail classes in every run.

What do the results show?

7 The results, read table by table

This chapter goes through the paper's results in the order of its results section. For each table or figure it says which question it answers, how to read it, what it shows and what it does not. Unless a caption says otherwise, a cell is the mean ± standard deviation over seeds 3–5, a difference is FedECS-LT minus the other method, paired over the same seeds, and "significant" means $p < 0.05$ after Holm's correction (Section 6.6). The noise floor is 1.4 points without the readout and 0.7 with it.

Two labels recur. *FedECS-LT* is the full method, ending with the exact readout. *FedECS-LT (heads only)* is the same run before the readout, the heads as Stage III left them. Every other method appears *without the readout* in the main tables, as its authors defined it; in Section 7.4 the readout is added to the methods that can take it.

7.1 The main comparison

The question. Does FedECS-LT raise balanced accuracy over every published method when clients have different models and the training set has a long tail? This is RQ2.

How to read the tables. Table 3 is the paper's main table: one row per method, one column per benchmark, the best mean in each column in bold. Table 4 is its paired-difference table: each cell is FedECS-LT minus the row's method, with the 95% interval in brackets, bold where the difference is significant. A difference whose interval excludes zero but is not bold passed the uncorrected test and failed Holm's correction.

The numbers. On CIFAR-10-LT FedECS-LT reaches 52.6 ± 1.3, against 37.5 ± 1.2 for the strongest other method, FedDF: a margin of +15.1 points with Holm-corrected $p = 0.0052$. On CIFAR-100-LT it reaches 18.7 ± 0.2 against 10.0 ± 0.5 for FedDF, a margin of +8.6 with $p = 0.0042$. Against GeFL-F the differences are +18.9 [+16.3, +21.6] and +11.0 [+10.2, +11.8]. On CIFAR-100-LT no class has zero recall for all ten models of FedECS-LT (0), while 12.3 classes on average are recognised by none of the models of FedDF.

The heads alone. Before the readout the heads of FedECS-LT reach 42.8 ± 2.0 and 14.3 ± 0.6. That is +5.2 points ahead of FedDF on CIFAR-10-LT, with $p = 0.066$, not significant, and +4.3 ahead on CIFAR-100-LT, with $p = 0.021$, significant. The readout itself adds +9.8 and +4.4 points to the heads. Of the +15.1-point margin on CIFAR-10-LT, therefore, about a third comes from the heads and two thirds from the readout.

What it shows. The measured statement is narrow: the full method, with its readout, is above every published method of this setting as that method was published, by a large and significant margin on both benchmarks. The heads alone are also ahead, by less, and the CIFAR-10-LT part of that is not resolved by three seeds.

Table 3: Balanced accuracy (%), the paper's main table. Look at the first three rows: the full method, its heads before the readout, and GeFL-F; then at the strongest other method in each column.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**52.6 ± 1.3**	**18.7 ± 0.2**
FedECS-LT (heads only)	42.8 ± 2.0	14.3 ± 0.6
GeFL-F	33.7 ± 1.2	7.7 ± 0.4
Balanced Softmax	36.3 ± 1.2	9.0 ± 0.3
FedLC	34.3 ± 1.1	8.5 ± 0.3
FedRS	34.1 ± 1.1	8.5 ± 0.3
FedDF	37.5 ± 1.2	10.0 ± 0.5
Local	32.1 ± 0.7	7.9 ± 0.8
FedProto	27.1 ± 0.8	4.8 ± 0.3
FedTGP	25.3 ± 1.2	4.5 ± 0.6
FedGen	26.9 ± 2.0	4.5 ± 1.3
FedGH	31.9 ± 1.1	6.5 ± 0.1
LG-FedAvg	31.3 ± 0.8	6.3 ± 0.1
FedMRL	33.7 ± 1.0	8.3 ± 0.2

What it does not show. It does not show that the generator side of FedECS-LT is what helps. Every other method in this table lacks the readout, and the readout is a known kind of classifier (Section 5.5) that can be added to most of them. The comparison that answers this follows in Section 7.4.

7.2 The other model-heterogeneous methods

The question. Do the methods that share prototypes, a generator of embeddings or a shared sub-network do better than the generator-based ones under a long tail?

The answer. No. In Table 3 FedProto, FedTGP, FedGen, FedGH, LG-FedAvg and FedMRL are close to or below Local. None of them gives an architecture any information about the classes its own clients lack: a prototype of class 9 is only useful to a head that already predicts class 9. FedProto, FedTGP and FedGen also decline during training. They are best in the first rounds and lose 5.7 ± 1.0, 6.7 ± 0.7 and 5.3 ± 0.9 points by the last round on CIFAR-10-LT. The heads of FedECS-LT decline too, by 3.4 ± 1.2 points on CIFAR-10-LT. The paper's appendix gives every method's best round, the measure GeFL reports; the readout is computed once, after the last round, so it has no best round.

What it does not show. These methods run as ports onto the frozen extractor with settings from their library, meant for end-to-end training (Section 6, threats). Their low numbers may partly be the port. The paper did not run a check of the ports.

7.3 Accuracy on the clients' own class mixes

The question. Balanced accuracy weights every class equally. What happens to the accuracy each client actually sees, on test images drawn with its own class frequencies?

The numbers. FedECS-LT reaches 59.2 ± 1.8 on CIFAR-10-LT and 28.3 ± 0.3 on CIFAR-100-LT, and its heads alone 68.6 ± 1.9 and 31.5 ± 0.3. GeFL-F reaches 73.0 ± 2.3 on

Table 4: FedECS-LT minus each method (points), paired over seeds 3–5, with the 95% t interval; bold: significant after Holm's correction. The paper's paired-difference table. Look at the smallest difference in each column: it belongs to the strongest other method.

Method	CIFAR-10	CIFAR-100
GeFL-F	**+18.9 [+16.3, +21.6]**	**+11.0 [+10.2, +11.8]**
Balanced Softmax	**+16.3 [+13.5, +19.0]**	**+9.7 [+8.9, +10.5]**
FedLC	**+18.3 [+16.2, +20.5]**	**+10.2 [+9.4, +11.0]**
FedRS	**+18.5 [+16.4, +20.6]**	**+10.2 [+9.5, +11.0]**
FedDF	**+15.1 [+12.6, +17.5]**	**+8.6 [+7.3, +10.0]**
Local	**+20.5 [+17.2, +23.9]**	**+10.7 [+8.6, +12.8]**
FedProto	**+25.5 [+24.2, +26.8]**	**+13.9 [+13.5, +14.4]**
FedTGP	**+27.3 [+25.4, +29.2]**	**+14.2 [+13.0, +15.4]**
FedGen	**+25.7 [+23.4, +28.1]**	**+14.1 [+10.5, +17.8]**
FedGH	**+20.7 [+18.1, +23.3]**	**+12.1 [+11.8, +12.5]**
LG-FedAvg	**+21.3 [+19.2, +23.4]**	**+12.4 [+12.1, +12.7]**
FedMRL	**+18.9 [+16.8, +21.0]**	**+10.4 [+9.9, +10.9]**

CIFAR-10-LT and Local 72.7 ± 2.6. FedECS-LT is below 11 of the 11 other federated methods on CIFAR-10-LT and 7 of 11 on CIFAR-100-LT.

Why. This is the price of balanced accuracy and of the readout, and it follows from their definitions. A client whose test images are mostly class 0 is best served by a last layer that favours class 0. Logit adjustment and the readout's equal class priors remove exactly that preference. The readout, which treats all classes as equally likely, lowers this metric by about nine points on CIFAR-10-LT. A deployment that cares about each client's own accuracy could add each client's own log-prior back to the readout's biases at prediction time, the reverse of Section 4.2; the paper did not test this.

7.4 The same readout for every method

The question. How much of FedECS-LT's margin remains when every method of the GeFL-F pipeline gets the same readout? This is RQ3, and its answer decides how the paper's claim must be worded.

Why this is the fair comparison of the bodies. With the readout, every last layer is replaced by the same estimator, computed from the same kind of pooled statistics, through the same procedure. What still differs between two methods is the body that produced the embeddings. So the difference with the readout measures what FedECS-LT's other components (the anchored generator, the moment correction, consensus soft labels, logit adjustment and ten synthetic epochs) do to the bodies, read through one linear classifier. The comparison without the readout measures the methods as published. A reader needs both: the first says whether FedECS-LT beats what exists; the second says whether its parts beyond a known classifier contribute.

The numbers. With the readout (Tables 5 and 6) FedECS-LT is +2.3 points [-0.4, +5.0] ahead of GeFL-F on CIFAR-10-LT and +3.8 [+3.1, +4.5] on CIFAR-100-LT. Against the strongest other method with the readout, Balanced Softmax on CIFAR-10-LT and FedRS on CIFAR-100-LT, the margins are +1.4 [-0.7, +3.6] and +1.5 [+0.6, +2.4], with Holm-corrected

Table 5: Balanced accuracy (%) with every last layer set by the exact readout, the paper's readout table. Methods of the GeFL-F pipeline only. Compare each row with its value in Table 3: the readout raises every method.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**52.6 ± 1.3**	**18.7 ± 0.2**
GeFL-F	50.3 ± 1.2	14.9 ± 0.2
Balanced Softmax	51.2 ± 0.7	17.2 ± 0.3
FedLC	51.2 ± 0.7	17.2 ± 0.3
FedRS	51.1 ± 0.7	17.2 ± 0.3
FedDF	51.0 ± 1.2	15.8 ± 0.3

Table 6: FedECS-LT minus each method, both with the readout (points), paired over seeds 3–5, with the 95% t interval; bold: significant after Holm's correction. From the paper's appendix. Look at whether the intervals contain zero.

Method	CIFAR-10	CIFAR-100
GeFL-F	+2.3 [-0.4, +5.0]	**+3.8 [+3.1, +4.5]**
Balanced Softmax	+1.4 [-0.7, +3.6]	+1.5 [+0.6, +2.4]
FedLC	+1.4 [-0.6, +3.5]	+1.5 [+0.6, +2.4]
FedRS	+1.5 [-0.6, +3.5]	+1.5 [+0.6, +2.4]
FedDF	+1.6 [-1.1, +4.3]	**+2.9 [+2.1, +3.6]**

$p = 0.37$ and 0.058. On CIFAR-100-LT the uncorrected interval excludes zero but the corrected test does not reach 0.05. The differences to GeFL-F and FedDF on CIFAR-100-LT stay significant ($p = 0.0095$ and 0.014).

What it shows. Three things. First, the readout raises every method by a large amount: Balanced Softmax gains +14.8 points on CIFAR-10-LT, GeFL-F +16.6. Second, once every method has it, the methods are close. Third, Balanced Softmax with the readout reaches 51.2 ± 0.7 on CIFAR-10-LT. With one client per architecture, Balanced Softmax trains every head on its own client's data with no generator and no exchange after Stage I. So a body trained alone, plus the federated readout, comes within +1.4 points of the full method. Most of the margin of Section 7.1 is the readout's.

What it does not show. It does not show that the other components do nothing: a gain of 1.5 points with three seeds and a noise floor of 0.7 is neither shown nor ruled out. It also does not show the components' effect without the readout, which is the subject of the pending ablations (Section 7.10). And because the readout is one fixed linear classifier, a body that is better for another last layer would not show it here.

7.5 Rare classes

The question. Where among the classes does the gain sit? If FedECS-LT addresses the long tail, the gain should be largest on the rare classes.

How to read it. Table 7 gives the mean recall of the tail classes (the 40% least frequent: classes 6–9 on CIFAR-10-LT, 60–99 on CIFAR-100-LT). Figure 6 plots each class's recall against its number of training images on CIFAR-100-LT.

Table 7: Recall (%) of the tail classes, the 40% least frequent; FedECS-LT with its readout, other methods without it. From the paper's tail table. Compare FedECS-LT with its heads only and with GeFL-F.

Method	CIFAR-10	CIFAR-100
FedECS-LT	**50.3 ± 1.5**	**8.2 ± 0.2**
FedECS-LT (heads only)	31.4 ± 2.7	6.4 ± 0.3
GeFL-F	13.9 ± 1.2	1.7 ± 0.1
Balanced Softmax	20.0 ± 1.9	2.2 ± 0.1
FedLC	16.1 ± 1.4	1.7 ± 0.1
FedRS	15.7 ± 1.4	1.7 ± 0.1
FedDF	10.6 ± 1.0	0.8 ± 0.1
Local	14.0 ± 1.2	1.6 ± 0.2
FedProto	10.7 ± 1.9	0.9 ± 0.0
FedTGP	10.8 ± 1.7	1.2 ± 0.5
FedGen	8.6 ± 1.1	0.7 ± 0.3
FedGH	13.3 ± 0.8	1.1 ± 0.1
LG-FedAvg	13.3 ± 0.6	1.0 ± 0.0
FedMRL	14.9 ± 1.2	1.7 ± 0.1

The numbers. Against GeFL-F on CIFAR-10-LT, FedECS-LT gains +36.4 points on the tail classes, +11.8 on the medium ones and +2.7 on the head classes. Its tail recall is 50.3 ± 1.5 against 13.9 ± 1.2. Against FedDF it gains +39.7 on the tail and loses -8.4 on the head classes. On CIFAR-100-LT the gain over GeFL-F is spread over the three groups (head +12.6, medium +15.4, tail +6.5). There the tail recall is 8.2 ± 0.2 against 1.7 ± 0.1, and the worst-class recall 0.2 ± 0.2 against 0.0 ± 0.0: some class is still lost on CIFAR-100-LT.

With the readout for every method. The tail gain over the loss-only methods disappears. FedECS-LT minus Balanced Softmax, both with the readout, is -0.03 points of tail recall on CIFAR-10-LT and -0.1 on CIFAR-100-LT. Against GeFL-F with the readout a tail gain remains, +2.8 [-0.2, +5.8] on CIFAR-10-LT and +1.8 [+1.3, +2.3] on CIFAR-100-LT, significant on CIFAR-100-LT ($p = 0.021$).

What it shows and does not. The long-tail gain is real and sits where it should, on the rare classes; its trade is head-class recall against FedDF. The same-readout numbers say where it comes from: from the last layer. The part that remains against GeFL-F is consistent with the generator side helping the bodies of GeFL-F's own pipeline, but against a body trained without any generator it is zero.

7.6 Natural class imbalance

The question. Does the result carry over to datasets whose imbalance is their own, which played no part in the development of the method? This is the first half of RQ4, and it is the cleanest test in the paper (Section 6.7).

The numbers. Without the readout for the others (Table 8), FedECS-LT has the highest mean on all six datasets. Its margin over the strongest other method runs from +6.4 (OrganAMNIST) to +18.4 points (PathMNIST), significant on three of the five datasets with three seeds; EMNIST (ByClass) has one seed, so no test. With the readout for every method

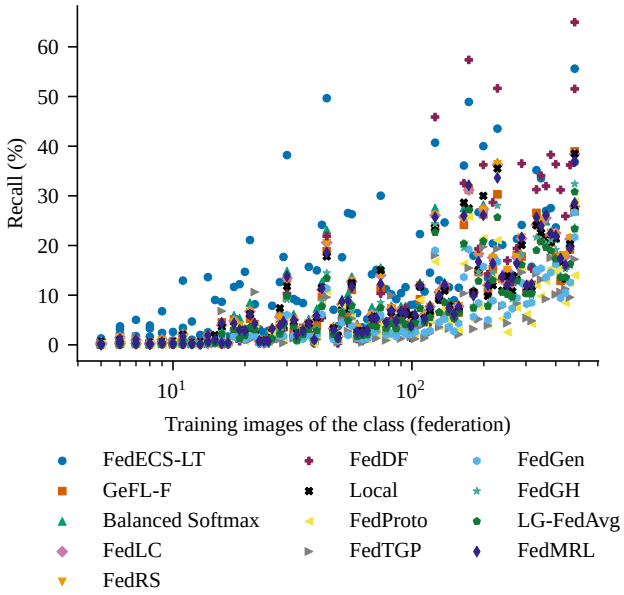


Figure 6: Recall of each class of CIFAR-100-LT against its number of training images in the federation (logarithmic axis); each point is one class, mean over seeds 3–5. Look at the left side of the plot, the rarest classes, where the methods separate.

of the pipeline (Table 9), the margins run from -0.03 (BloodMNIST, against FedDF) to +0.9 (FER-2013). None is significant after Holm's correction (the smallest p, on OrganAMNIST, is 0.051). All but the BloodMNIST margin exceed the noise floor of 0.7 points, which is necessary but not sufficient for a resolved difference. On BloodMNIST, FedDF with the readout has the higher mean. The readout changes FedECS-LT itself by +11.6 points on PathMNIST and by -0.3 on OrganAMNIST, where it slightly lowers the heads' accuracy. On FER-2013 the gate chose the generator with class rows.

What it shows. On data the development never saw, the pattern of the CIFAR benchmarks repeats: a large margin as published, a small and unresolved one once every method has the readout. That the pattern repeats is evidence that the CIFAR result is not an artefact of tuning. That the same-readout margin stays small is evidence that the readout carries the result here too.

What it does not show. On OrganAMNIST, BloodMNIST and PathMNIST the realised imbalance ratio is 5 or less (Section 6), so these three test label skew more than a long tail.

7.7 One shared model

The question. When all clients can run the same model, how does FedECS-LT compare with the federated long-tail methods? This is the second half of RQ4. It is a test of scope, outside the setting the method was built for.

The numbers. FedYoYo, FedLoGe, CCVR, FedETF, FedAvg and MOON are above FedECS-LT on CIFAR-10-LT, and FedYoYo, FedLoGe and CCVR on CIFAR-100-LT (Table 10). FedECS-LT reaches 58.4 ± 1.6 on CIFAR-10-LT against 61.0 ± 1.8 for plain FedAvg and 78.1 ± 0.9 for FedYoYo; its heads alone reach 61.3 ± 1.8. The readout changes the heads by -3.0 points on CIFAR-10-LT and -0.1 on CIFAR-100-LT. The CIFAR-100-LT values of FedECS-LT

Table 8: Balanced accuracy (%) on the six naturally imbalanced datasets, the paper's natural-imbalance table; FedECS-LT with its readout, other methods without it. A value without ± is from one seed. Look at the gap between the first row and the rest.

Method	DermaMNIST	OrganAMNIST	BloodMNIST	PathMNIST	EMNIST (ByClass)	FER-2013
FedECS-LT	**45.4 ± 1.9**	**83.3 ± 1.3**	**86.3 ± 1.3**	**73.6 ± 0.4**	**67.3**	**39.2 ± 1.0**
FedECS-LT (heads only)	36.8 ± 0.8	83.7 ± 2.5	84.2 ± 2.0	61.9 ± 3.2	62.1	30.9 ± 1.5
GeFL-F	27.3 ± 0.3	76.9 ± 3.5	78.5 ± 2.2	55.2 ± 2.3	49.8	27.5 ± 0.2
Balanced Softmax	28.6 ± 0.1	68.1 ± 4.8	69.9 ± 3.8	54.6 ± 2.2	45.6	26.6 ± 0.4
FedLC	26.0 ± 0.7	63.9 ± 4.4	66.5 ± 3.9	51.9 ± 2.6	42.1	25.7 ± 0.2
FedRS	26.0 ± 0.6	63.5 ± 4.4	66.0 ± 3.9	52.6 ± 1.4	41.4	25.7 ± 0.3
FedDF	20.5 ± 1.4	64.7 ± 6.1	70.6 ± 7.5	48.2 ± 1.2	47.2	25.7 ± 0.3
Local	25.3 ± 0.5	63.5 ± 4.2	63.6 ± 3.6	49.3 ± 3.4	41.4	25.2 ± 0.5
FedProto	25.5 ± 1.5	58.6 ± 4.1	65.4 ± 4.5	54.0 ± 0.8	39.4	20.8 ± 0.5
FedTGP	24.7 ± 1.0	50.7 ± 3.9	59.5 ± 2.4	49.8 ± 1.3	34.8	20.6 ± 0.7
FedGen	26.0 ± 0.6	60.1 ± 5.1	64.1 ± 3.2	51.7 ± 0.5	32.2	24.4 ± 0.5
FedGH	27.1 ± 0.2	61.8 ± 4.2	65.9 ± 3.6	53.8 ± 1.2	38.7	24.7 ± 0.4
LG-FedAvg	26.8 ± 0.4	62.6 ± 4.3	65.9 ± 3.2	53.3 ± 1.2	38.5	24.7 ± 0.6
FedMRL	27.1 ± 0.3	64.2 ± 4.3	67.2 ± 3.5	53.7 ± 1.0	42.1	25.9 ± 0.5

Table 9: The same datasets with the exact readout for every method of the GeFL-F pipeline, from the paper's appendix. Look at how close the rows are.

Method	DermaMNIST	OrganAMNIST	BloodMNIST	PathMNIST	EMNIST (ByClass)	FER-2013
FedECS-LT	**45.4 ± 1.9**	**83.3 ± 1.3**	86.3 ± 1.3	**73.6 ± 0.4**	**67.3**	**39.2 ± 1.0**
GeFL-F	44.6 ± 1.6	82.5 ± 1.0	86.3 ± 0.4	72.3 ± 0.5	66.5	38.2 ± 0.7
Balanced Softmax	42.8 ± 0.8	79.0 ± 1.4	82.9 ± 1.2	70.6 ± 0.6	64.5	35.7 ± 0.8
FedLC	43.0 ± 0.9	79.1 ± 1.3	83.0 ± 1.0	70.7 ± 0.7	64.6	35.6 ± 0.7
FedRS	43.1 ± 0.9	79.0 ± 1.3	82.9 ± 1.0	70.8 ± 0.8	64.5	35.5 ± 0.7
FedDF	44.7 ± 1.1	82.1 ± 1.3	**86.4 ± 0.5**	72.8 ± 0.1	66.6	37.7 ± 0.7

are from one seed, because runs of FedECS-LT failed there where the baselines finished; the missing runs are not random.

Why the readout does not help here. The readout repairs a last layer fitted to one client's prior (Section 4.2). With one shared model, the head is trained by every client in turn and averaged by FedAvg, so its last layer already sees every client's classes. There is nothing left for the readout to repair, and a linear discriminant on non-Gaussian embeddings can be worse than a trained last layer. The paper states that it did not test why the readout lowers the accuracy.

Why the generator does not help here. With one shared model, FedAvg itself carries every client's knowledge into every client's model, which is the job the generator does when models differ. FedECS-LT also trains its head on frozen features of unaugmented images, while the baselines train the whole network end to end on augmented images (Section 6). The paper offers the second difference as a possible explanation and states that it did not test it.

The published numbers. The baselines run in their authors' library on the paper's splits and report the model after the last round. Against FedYoYo's own evaluation in the same protocol, most of them are within about three points of the published values. Three (CReFF, Fed-GraB, FedLC) are further below; they run with the library's common settings, which differ from those of their own papers, and were not tuned.

What it shows. FedECS-LT is a method for clients with different models. When the clients can share one model, a plain FedAvg is better on CIFAR-10-LT, and the long-tail methods

Table 10: Balanced accuracy (%) with one ResNet-8 shared by all clients (FedLTLib protocol), imbalance ratio 100; the paper's shared-model table. A value without ± is from one seed. Look at how many rows are above the first one.

Method	CIFAR-10	CIFAR-100
FedECS-LT	58.4 ± 1.6	34.0
FedECS-LT (heads only)	61.3 ± 1.8	34.1
FedLC	58.3 ± 1.9	32.8 ± 0.1
FedAvg	61.0 ± 1.8	33.2 ± 0.3
FedProx	54.4 ± 1.6	31.2 ± 0.4
MOON	59.9 ± 1.9	33.4 ± 0.2
CCVR	68.0 ± 1.2	35.8 ± 0.1
CReFF	56.8 ± 2.3	28.1 ± 0.0
FedETF	65.3 ± 4.3	32.8 ± 0.2
Fed-GraB	49.4 ± 2.4	27.8 ± 0.1
FedLoGe	66.7 ± 2.9	39.4
FedYoYo	**78.1 ± 0.9**	**42.6 ± 0.5**
RUCR	54.0 ± 1.6	32.9 ± 0.2

built for that setting are much better.

7.8 Representations

The question. Do the bodies of FedECS-LT separate the classes better than those of GeFL-F, independently of any last layer?

The measurement. The leave-one-out 10-nearest-neighbour accuracy of the test embeddings classifies each test image by the majority class of its ten nearest other test embeddings. It uses no last layer at all. It is 40.9% for FedECS-LT, 37.7% for GeFL-F and 38.9% for FedDF on CIFAR-10-LT, averaged over the ten architectures and three seeds. FedECS-LT is higher in 29 of the 30 architecture–seed pairs against GeFL-F and in 24 against FedDF. Figure 7 shows a t-SNE map of the embeddings of one architecture.

What it shows and does not. The bodies of FedECS-LT separate the classes a little better, consistently across architectures and seeds. The difference is a few points, which agrees with the small same-readout margin of Section 7.4. A t-SNE map is an illustration; it cannot carry a quantitative claim.

7.9 Calibration, the weakest architecture and cost

Calibration. The expected calibration error of FedECS-LT is 21.2 ± 0.5 on CIFAR-10-LT, against 48.0 ± 2.2 for GeFL-F and 35.9 ± 1.9 for FedDF; it ranks 3 of thirteen methods there. On CIFAR-100-LT it ranks 5, with 43.8 ± 0.4 against 42.4 ± 0.7 for GeFL-F and 31.8 ± 1.2 for FedDF. So calibration is not an advantage on CIFAR-100-LT.

The weakest architecture. Balanced accuracy averages ten models. The weakest architecture of FedECS-LT reaches 48.5 ± 1.5 on CIFAR-10-LT and 14.9 ± 1.9 on CIFAR-100-LT, so the mean does not hide a failed model.

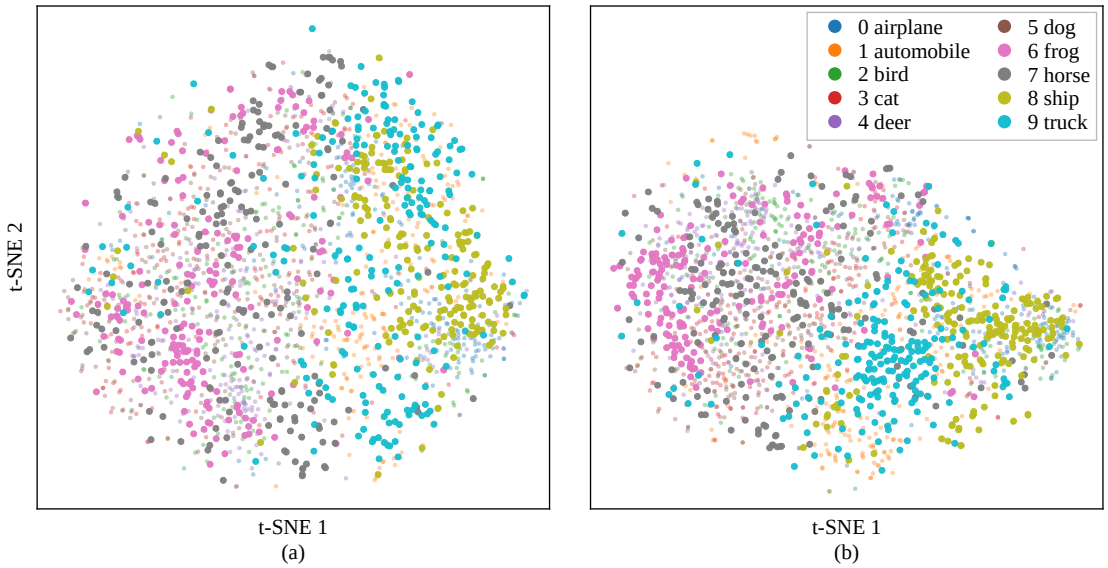


Figure 7: t-SNE of the test embeddings of one architecture on the same 2,000 CIFAR-10-LT test images, (a) GeFL-F and (b) FedECS-LT; tail classes drawn larger. Look only at which points group together: in a t-SNE map the sizes of clusters and the distances between them carry no meaning.

Cost. Table 11 gives the measured cost. FedECS-LT takes 1.13 times the wall-clock time of GeFL-F and 1.30 times its communication. Most of the extra time is the longer synthetic phase ($T_s = 10$), and most of the extra communication is the generator, which is larger than GeFL-F's because of its projection of the class means. Against Balanced Softmax, which trains its heads locally, FedECS-LT communicates 185 times as much and takes 15 times as long, for +1.4 points when both have the readout. This is the trade a reader must weigh, and Section 9 returns to it.

7.10 The experiments still pending

Five experiments asked for by the reviews were built as Flower notebooks and had not returned when this text was written. The paper's text for each is already written and appears by itself once the results are filled in. For each, here is the question, what result would support or weaken the paper, and what changes.

Several clients per architecture. The paper's 50-client setting has five clients per architecture, so heads of one architecture are averaged in Stage III. The question: is the readout's dominance an artefact of one client per architecture? With averaging, Balanced Softmax's heads would no longer be purely local, and their last layers would no longer fit one client's prior, so the readout should matter less. The paper is supported if FedECS-LT stays ahead of every method with the readout by more than the noise floor; it is weakened if the margins fall to zero, because then FedECS-LT's case would rest on the one-client-per-architecture setting alone. The runs are pending; the first attempt on CIFAR-10-LT ran out of host memory on Kaggle (run log) and was rebuilt to run one job per GPU.

New client splits. Seeds 6–8 draw splits that the development never used, for FedECS-LT, GeFL-F, FedDF and Balanced Softmax. The question: are the CIFAR margins the product of choosing settings on the reported splits? The paper is supported if the margins on seeds 6–8

Table 11: Cost on CIFAR-10-LT, from the paper's cost table: communication per client over the whole training, wall-clock time of a run (on a shared T4 GPU, so indicative only), forward MFLOPs per image and training GFLOPs per round. Compare the first row with Balanced Softmax.

Method	Comm./client (MB)	Time (s)	MFLOPs/img	Train. GFLOPs/round
FedECS-LT	11,278 ± 113	7,910 ± 2,253	**2.45**	1,424 ± 112
GeFL-F	8,659	7,025 ± 446	**2.45**	579 ± 42
Balanced Softmax	60.8	542 ± 99	**2.45**	**485 ± 34**
FedLC	60.8	501 ± 118	**2.45**	**485 ± 34**
FedRS	60.8	**417 ± 131**	**2.45**	**485 ± 34**
FedDF	8,659	5,050 ± 2,736	**2.45**	579 ± 42
Local	-	536 ± 98	-	-
FedProto	18.8	4,025 ± 109	2.45	485 ± 34
FedTGP	18.8	4,121 ± 218	2.45	485 ± 34
FedGen	128	4,443 ± 1,033	2.45	485 ± 34
FedGH	18.8	3,149 ± 103	2.45	485 ± 34
LG-FedAvg	18.8	2,890 ± 212	2.45	485 ± 34
FedMRL	**17.1**	4,545 ± 804	3.84	871 ± 34

are like those on 3–5, and weakened if the same-readout margin vanishes there. The runs are pending.

Variants of GeFL-F. Seven variants isolate single changes, among them GeFL-F with decoupled weight decay alone (AdamW) and GeFL-F and FedDF with ten synthetic epochs. The AdamW arm is the direct test of the decay diagnosis of Section 4.1: if it raises GeFL-F's balanced accuracy or tail recall by more than the noise floor, the class row shrinkage is shown to cost accuracy; if not, the shrinkage is a property of the generator without a measured consequence. The ten-epoch arms test whether the heads' advantage is computation: if GeFL-F or FedDF with ten synthetic epochs reach FedECS-LT's heads, it is. The runs are pending.

Components of FedECS-LT. Each run removes one component: the mean anchor, CCWA, the moment correction, consensus soft labels, logit adjustment, or nine of the ten synthetic epochs. One more replaces the trained generator by Gaussians with the exact class means and covariances, which tests whether a trained generator is needed at all. The question is RQ5. Each component is shown to matter if removing it lowers balanced accuracy with the readout by more than 0.7 points, significantly. If all removals stay within the floor, no component has a measured effect beyond the readout, and the paper must say so. If the Gaussian sampler matches the full method, Stage II is unnecessary. The runs are pending.

Other settings. Sweeps over the label skew ($\alpha = 0.1$ and 1.0), the number of clients (50 and 100) and the imbalance ratio (50 and 10) on CIFAR-10-LT test whether the result depends on the main setting's values. Their runs are pending, and the paper's text for them is not yet written.

What, taken together, does this evidence support?

8 What the evidence supports, and its limits

What is established. In generator-based learning with different client models and a long tail, the runs show three properties of GeFL-F: its generator's rare class rows are shorter, its generated features carry little class information, and its last layers lose most of what an exact readout recovers (Section 4). The runs show that FedECS-LT, ending with that readout, has a higher mean than every published method of the setting on both CIFAR benchmarks and on all six naturally imbalanced datasets, by margins that are large and significant on both CIFAR benchmarks and on three of the five natural datasets with three seeds; on CIFAR-10-LT the gain is mostly in the recall of the rare classes (Sections 7.1 and 7.6). They also show that most of this margin is the readout's: added to the other methods of the GeFL-F pipeline, it leaves FedECS-LT a lead of +1.4 and +1.5 points on the CIFAR benchmarks and at most +0.9 on the natural datasets, which three seeds do not resolve (Section 7.4). The scope of all of this is one client per architecture, a small frozen extractor shared by all clients, a label shift between clients, and three seeds.

The limits of the study. The paper's limitations section lists what the study does not cover. Each item limits what may be claimed.

- **A small, frozen, shared extractor.** The architectures differ only in their heads. The results say nothing about pre-trained or larger extractors, or about clients whose whole networks differ. A stronger extractor would raise every method, and whether it would change the ranking is not known.
- **No method built for this exact problem exists.** The comparison is with long-tail losses applied per client, with FedDF, and with model-heterogeneous methods designed for label skew without a long tail. A margin over methods built for another problem is weaker evidence than a margin over a direct competitor would be.
- **Label shift only.** The splits differ between clients only in their class proportions (Section 3). The key fact behind the method, that every client's class-c samples point at the same target, fails under feature shift (the same class looks different on different clients) or concept shift. Those cases are not tested, and the exactness argument does not cover them.
- **One client per architecture** in every reported result. The 50-client runs are pending (Section 7.10).
- **No differential privacy.** The method can be called privacy-preserving only in the sense of secure aggregation against an honest-but-curious server (Section 5.7).
- **Cost.** FedECS-LT costs more than GeFL-F and far more than per-client long-tail losses (Table 11).
- **Low absolute accuracy.** GeFL's own authors attribute GeFL-F's lower accuracy on CIFAR-10 to its small extractor and heads. In GeFL's balanced setting the paper's earlier implementation of GeFL-F reaches 56.8% on CIFAR-10, close to GeFL's published value of about 55.9%, so the low numbers are those of the setting, not of a faulty re-implementation. They are not comparable with centralised long-tail results for ResNet-32 and larger networks.

The trade-offs. Four costs come with the method, and a reader deciding whether to use it should weigh each.

- **Each client's own accuracy.** Balanced accuracy and the readout's equal priors serve the federation's rare classes at the expense of each client's own frequent ones. On the clients' own class mixes FedECS-LT is below most other methods (Section 7).
- **Communication and time.** 185 times the communication and 15 times the time of Balanced Softmax on CIFAR-10-LT, for a same-readout margin of +1.4 points that is not resolved.
- **Every client runs every head.** The consensus needs every client to run every architecture's head on the generated pool each round, and the readout every body once. This is forward computation only, no training, but it assumes every client can store and run the largest head. The motivation of model heterogeneity, that clients run different networks because their hardware differs, is therefore only partly kept: the clients still train different networks, but each must be able to evaluate all of them. The architectures are not secret.
- **Released class means.** The server and every client learn the federation's class means and spreads. For a class held by one client, that is the client's own statistic; for a small class, the mean says more about each image. With a shallow extractor a class mean is close to a mean image at half resolution.

What follows and what does not. It follows that the exact readout raises every method of this setting to which it was added, by far more than the other differences between them, for one extra round of communication. It follows that FedECS-LT as a whole has the highest measured balanced accuracy when clients have different models. It does not follow that exact class statistics in the generator are what makes FedECS-LT work: that is the question of the pending ablations. It does not follow that the decay diagnosis is a cause: that is the question of the AdamW run. And it does not follow that the result holds with several clients per architecture.

Which questions will a reviewer ask, and what are the honest answers?

9 Defending the paper

Three internal reviews of the paper raised the objections below. Each is stated as a reviewer would put it, followed by the honest answer, the evidence, and the experiment that would settle it with its status. An honest answer concedes what the evidence does not show; a reviewer who finds a concession in the paper is less likely to find an overclaim.

1. "Isn't the whole gain just the readout?"

Answer. Mostly, yes, and the paper says so. Without the readout for the others, FedECS-LT leads the strongest other method by +15.1 and +8.6 points. With the readout for every method of the GeFL-F pipeline it leads by +1.4 [-0.7, +3.6] and +1.5 [+0.6, +2.4], not significant after Holm's correction ($p = 0.37$ and 0.058), and by at most +0.9 on the natural datasets

(Sections 7.4 and 7.6). Balanced Softmax, which has no generator, comes within +1.4 points once it has the readout.

Why the readout is still part of the method. The readout follows from the diagnosis: Section 4.2 shows that no per-client correction of the loss can give a last layer information about absent classes, and the readout is the repair that does. The paper therefore defines FedECS-LT as ending with it, credits the family it belongs to (Fed3R, FedNCM, FedCOF), and reports every method of the pipeline with it added, so the reader can see exactly how much is the readout's. What the paper must not claim is that the generator side has a demonstrated effect beyond the readout.

Settled by. The component ablation (RQ5) and the variants of GeFL-F, both pending (Section 7.10).

2. "The name says exact class statistics, but half the components do not use them"

Answer. Correct, and the paper now says which do. Three components use the secure statistics: the anchored generator (or CCWA), the moment correction and the readout. Logit adjustment uses each client's own class frequencies, consensus soft labels use the heads' predictions, and ten synthetic epochs are computation (Section 5.6). The name describes the first group and the readout, which carries most of the effect.

3. "Is the weight-decay diagnosis causal?"

Answer. Not shown. What is measured is that the rare classes' rows are several times shorter (Fig. 2), as the contraction lemma predicts. Against a causal reading: GeFL-F's generated features are recognised at 12.9 ± 2.4 on CIFAR-10-LT, 9.4 ± 0.9 on the head classes and 16.8 ± 0.3 on the tail classes, and at chance on CIFAR-100-LT (1.1 ± 0.1). If the shrinkage were what loses the rare classes, the head classes, whose rows are long, would be recognised; they are not. The row lengths also do not fall monotonically with class size. The step-count heuristic κ_c was not checked against the rows. The paper calls the shrinkage a measured property of the generator, not a demonstrated cause.

Settled by. GeFL-F with decoupled weight decay alone (AdamW), pending. If it raises GeFL-F's tail recall by more than the noise floor, the diagnosis gains a causal test; if not, the paper should say the shrinkage has no measured consequence for accuracy.

4. "The other diagnoses: do generated features without labels, and the moment correction, matter?"

Answer. The label-free lemma is exact, but its size in the pipeline is not settled. The synthetic phase does reduce the logit spread, and less for FedECS-LT than for GeFL-F; but FedDF, on GeFL-F's own generator, loses less spread than FedECS-LT (Section 4.3), so the spread change is not decided by whether the features carry their label. The moment correction brings the generated spreads to 0.73 to 0.98 times the exact ones (Section 5.3), but its fidelity result is mixed: on CIFAR-10-LT it raises the mean fidelity from 25.9 ± 2.1 to 28.9 ± 4.6 and

lowers the tail classes' from 44.1 ± 5.0 to 40.3 ± 2.8; on CIFAR-100-LT the mean is unchanged (8.8 ± 0.4). Its value for accuracy is unmeasured.

Settled by. The component ablation ("w/o moment correction", "w/o consensus soft labels"), pending.

5. "Were the settings tuned on the test sets?"

Answer. The development used the same splits (seeds 3–5) and the same test sets as the reported runs, comparing about 36 configurations on CIFAR-10-LT and 17 on CIFAR-100-LT; the baselines were not tuned (Section 6.7). The CIFAR margins may therefore favour FedECS-LT. Two things limit the damage. The six naturally imbalanced datasets were never used in development, and there the pattern repeats: large margins without the readout for the others, small ones with it (Section 7.6). And every setting is the same on every dataset; nothing was set per dataset.

Settled by. The run on new splits, seeds 6–8, pending.

6. "Is the heads' advantage just more computation and ensemble distillation?"

Answer. It may be. FedECS-LT trains ten synthetic epochs per round against one for GeFL-F and FedDF, with consensus soft labels, which are a form of ensemble distillation; its training costs about two and a half times the GFLOPs per round of GeFL-F (Table 11). The heads-only margin over FedDF, +5.2 on CIFAR-10-LT, is not significant ($p = 0.066$) and has not been separated from this extra computation.

Settled by. GeFL-F and FedDF with ten synthetic epochs, and FedDF with logit adjustment, pending.

7. "With one client per architecture nothing is averaged; does the result hold with several?"

Answer. Unknown. In every reported result each architecture has one client, so Stage III averages nothing, and the per-client loss methods become purely local training. That is exactly the case in which the readout helps most, because each last layer then fits one client's prior. With several clients per architecture, heads are averaged and the readout may matter less.

Settled by. The 50-client runs (five per architecture), pending; their first attempt ran out of host memory and was rebuilt.

8. "If every client runs every head, why have different models at all?"

Answer. The clients still *train* different networks; but the consensus needs each client to run every architecture's head on the generated pool every round, and the readout every body once. This is forward computation only, and the paper states it as an assumption of the setting (Section 8). It does weaken the hardware motivation of model heterogeneity: a client must be able to store and evaluate the largest head. It also means the architectures are not secret.

9. "Why is it worse than FedAvg with one shared model?"

Answer. Because the setting removes the problem FedECS-LT addresses. With one shared model, FedAvg carries every client's knowledge into every model, so a generator is not needed; and the last layer is trained by every client, so the readout has nothing to repair and lowers accuracy (-3.0 points on CIFAR-10-LT). FedECS-LT also trains its head on frozen features of unaugmented images while the baselines train end to end with augmentation. FedYoYo, FedLoGe, CCVR, FedETF, FedAvg and MOON are above it on CIFAR-10-LT (Section 7.7). The paper reports this protocol as a test of scope, and the honest conclusion is that FedECS-LT is a method for clients with different models only. Why the readout lowers accuracy here was not tested.

10. "Is a small gain worth 185 times the communication?"

Answer. Against Balanced Softmax with the readout, on CIFAR-10-LT, the measured gain is +1.4 points, not resolved, for 185 times the communication and 15 times the time. On this evidence, a practitioner who only needs balanced accuracy should use per-client Balanced Softmax plus the exact readout, which needs no generator. FedECS-LT's extra cost is justified only if the pending ablations show that its generator side adds accuracy, or in settings, such as several clients per architecture, where it may add more. Against GeFL-F, the base it extends, the overhead is 1.30 times the communication and 1.13 times the time.

11. "What do the released statistics leak?"

Answer. Secure aggregation hides each client's own sums, not the federation's. The server and every client learn each class's mean and spread. For a class held by one client these are that client's statistics; for a small class each image moves the mean noticeably; and with a two-convolution extractor a class mean is close to a mean image at half resolution (Section 5.7). The paper did not test an inversion of the means and has no differential privacy, so it claims nothing beyond secure aggregation against an honest-but-curious server.

Settled by. An inversion test on the smallest classes, or a differentially private release with its accuracy cost; neither is planned.

12. "Why are the absolute accuracies so low?"

Answer. Because the setting is GeFL-F's: a two-convolution extractor with 3 and 10 channels, frozen after Stage I, and small heads, under a long tail with ten clients. GeFL's authors attribute GeFL-F's lower accuracy to this extractor and these heads. In GeFL's own balanced setting the paper's earlier implementation of GeFL-F reaches 56.8% on CIFAR-10, against about 55.9% in GeFL's paper, so the implementation is not the cause. A stronger or pre-trained extractor was not studied.

Settled by. The reference trained on all data pooled, planned for GeFL's 50-client setting and pending, would give the ceiling of this extractor.

13. "How is this different from FedCOF, Fed3R or CCVR?"

Answer. The readout belongs to the same family as Fed3R (ridge regression), FedNCM (nearest class means) and FedCOF (an LDA-type classifier); these also note that their sums do not depend on the partition. The difference is where it is applied: per architecture, on bodies trained in the federation rather than pre-trained, with statistics from every client, including clients of other architectures, which requires every client to run every body. CCVR samples features from Gaussians with the pooled class moments to retrain a shared classifier; the moment correction instead maps a learned generator's samples onto those moments. The pending "w/ Gaussian sampler" run compares the two directly. The paper does not claim the readout or partition invariance as new.

14. "Why only three seeds?"

Answer. Compute: each configuration takes hours on a Kaggle GPU, and the study has several hundred runs. Three seeds give little power. A sign-flip permutation test cannot go below $p = 0.25$ with three pairs, so every significant result rests on the t-test's normality assumption, and a difference of one to two points cannot be resolved against a noise floor of 0.7 (Section 6.6). The paper therefore reports intervals, applies Holm's correction, calls small differences unresolved rather than absent, and adds three new seeds for the main comparison (pending).

The position to defend

The defensible claims are: three measured properties of GeFL-F under a long tail; a method that, ending with an exact readout, has the highest balanced accuracy of the setting by a large margin; and the finding that the readout, added to every method, carries most of that margin. Every other claim waits on a pending run, and the paper says which.